

A survey of non-learning-based abstractions for sequential decision-making

Robin Schmöcker
Faculty of EECS
Leibniz University Hannover
Hannover, Germany
schmoecker@tnt.uni-hannover.de

Alexander Dockhorn
Faculty of EECS
Leibniz University Hannover
Hannover, Germany
dockhorn@tnt.uni-hannover.de

Abstract—Abstraction techniques play a crucial role in enabling agents to make decisions more effectively by simplifying complex problems. This survey provides a comprehensive literature overview of non-learned abstraction construction methods and explores how these abstractions can enhance or be seamlessly integrated into existing solvers. We delve into key properties of abstractions, outline general strategies for constructing them, and discuss specialized approaches for specific problem domains, such as those with continuous action spaces. Additionally, we introduce the Abstraction Mapping Graph (AMG) framework, offering a structured lens through which abstraction usage can be systematically understood.

Index Terms—Artificial Intelligence, Abstractions, Sequential Decision-making.

I. INTRODUCTION

A plethora of important problems can be viewed as sequential decision-making tasks such as autonomous driving [82], energy grid optimization [114], financial portfolio management [18], or playing video games [112]. In general, the goal is to maximize an agent’s performance on these problems. One tool to improve the performance, which will be the focus of this survey, is the usage of abstractions that broadly speaking reduce a problem’s complexity. Usually, an abstraction itself is not a complete algorithm but rather a component that can be integrated to enhance other algorithms.

Though for many decision-making tasks learning-based algorithms beat non-learning-based approaches such as AlphaGo for Go [112] or OpenAI Five for Dota 2 [15], they are not always used in practice due to some undesirable properties they come with. For example, game studios rarely implement learning-based bots because they are unadaptable to changes in the game rules and would require costly retraining every time the game is altered in a significant way (e.g. during development or when patches are added). Even if the given decision-making problem is static, just a single training phase can be extremely resource-intensive, take weeks [112] or even months to complete [15], require experts to implement and tune, and produce an agent with a humanly uninterpretable behavior [146]. Therefore, it is worth studying non-learning-based algorithms and how these techniques can be enhanced using abstractions as they generally do not exhibit the above-mentioned unwanted properties.

This survey is structured as follows. First, in Section II, we will introduce the foundations needed for this survey which

includes the definition of a sequential decision-making task and what we consider abstractions. Then, in Section III, we will define the scope of this survey. Oftentimes, abstraction-using methods can be decoupled into two methods. One for constructing an abstraction and one for employing the abstraction to aid in the decision-making process. Usually, the latter does not depend on the concrete abstraction, making the abstraction interchangeable. Therefore, we split the construction and usage into two different sections. First, in Section IV, we will present different abstraction construction concepts (e.g. online versus offline abstraction construction) along with concrete methods that exemplify the respective concepts. Afterward, in Section V, we will give an overview of how abstractions can be applied to enhance a search or optimization algorithm. An overview of the presented papers whose entire pipeline is non-learning based is given in the Supplementary materials in Table I. Furthermore, the Supplementary materials contains a description of all benchmark problems that were either mentioned during this survey or appeared in any of the presented papers. The survey finishes with Section VI, in which we discuss possible future research avenues for abstraction techniques.

We remark that there already exist other surveys about the usage of abstractions in decision-making, however, they either focus on one particular type of abstraction [119], [138] or cover a larger scope than this survey at the cost of being comparatively broad and not covering the majority of non-learning based abstraction for sequential decision-making that qualified for this survey [46].

II. FOUNDATIONS

We use *partially observable stochastic games* (POSG) to model sequential decision-making problems.

Definition II.1. A POSG is a 7-tuple $(n, S, \mu_0, A, \mathcal{O}, \mathbb{P}, R)$ where the components are as follows:

- $n \in \mathbb{N}$ is the number of players.
- $S \neq \emptyset$ is the finite set of states.
- $\mu_0 \in \Delta(S)$ is the probability distribution for the initial state.

- $A = A_1 \times \dots \times A_n \neq \emptyset$ is the finite joint action space and for each $1 \leq i \leq n$, A_i is the action space for player i .
- $\mathcal{O} = \mathcal{O}_1 \times \dots \times \mathcal{O}_n \neq \emptyset$ is the finite set of joint observations.
- $\mathbb{P}: S \times A \mapsto \Delta(S \times \mathcal{O})$ is the stochastic transition function where we use $\mathbb{P}(s', o | s, a)$ to denote the probability of transitioning from $s \in S$ to $s' \in S$ and observing $o \in \mathcal{O}$ after taking action $a \in A$ in s .
- $R: S \times A \mapsto \mathbb{R}^n$ is the joint reward function with R_i being the reward function for player $1 \leq i \leq n$.

This is a slightly generalized definition than the one given by Bernstein et al. [61] where we use $\Delta(X)$ to denote the probability simplex of a finite, non-empty set X . For simplicity reasons, this definition only includes finite POSGs, however, this definition can be canonically generalized using probability measures to infinite state, observation, and action space problems which will appear in this survey.

Starting with $s_0 \sim \mu_0$, a POSG progresses from state s_t to s_{t+1} according to $\mathbb{P}$ given the current joint action decided by the players. Depending on the concrete problem setting, the game may end after $h \in \mathbb{N}$ steps (finite horizon) or continue indefinitely (infinite horizon) where each player's goal is to maximize one's own, possibly discounted, sum of obtained rewards. For decision-making, the agent $1 \leq i \leq n$ must only use o_i from the last observation $o \in \mathcal{O}$.

This POSG definition can model several special cases, for example, perfect information games can be obtained by setting $\mathcal{O} = S$ and $\mathbb{P}$ such that the observation and new state always coincide. *Partially Observable Markov Decision Processes* (POMDP) [125] are obtained by setting $n = 1$. Perfect information POMDPs are called *Markov Decision Processes* (MDP). Perfect recall agents can be modeled by letting each observation be the entire action and atomic observation history. Sequential games can be modeled by setting $\mathbb{P}(s_{t+1}, o_{t+1} | s_t, a_t) = \mathbb{P}(s_{t+1}, o_{t+1} | s_t, a_{t,i})$ if $t \bmod n = i - 1, 1 \leq i \leq n$ and state-dependent action spaces can be obtained by including the available actions in the observation and punishing an agent for choosing an illegal action such that an optimal strategy must only choose legal actions.

To avoid having to introduce an all-encompassing, certainly convoluted definition of what an abstraction of a POSG M is, we will only informally define it as another POSG M_{abs} with reduced complexity that is somehow related to the original POSG such information about the original can be inferred from the abstraction. Reduced complexity may for example mean that M_{abs} has a smaller state or action space, that M_{abs} is fully observable when M is not, or that M_{abs} is deterministic when M is not. A relationship between M_{abs} and M may be a map from an abstract action to an original action.

Before beginning with the actual survey, we would like to clarify some terms to be used throughout the remainder of this survey. In the context of tree searches, a node represents a state of an underlying POSG. If it is clear from the context or not of relevance, the term state may both refer to a node and an element in the original model's state space. Similarly, in the context of partially observable domains, we will refer to both an observation and a belief state as a state. Secondly, in general, the term grouping (be it states or actions), refers to abstractions with abstract states or actions that can formally be defined as the set of their constituent states or actions. Lastly, if we have to differentiate between the original model and an abstraction, we refer to the former's states and actions as the original states and actions and to the latter's as the abstracted states and actions.

III. SURVEY SCOPE

For a paper to qualify for Chapter IV where we present abstraction construction methods, it had to fulfill three criteria:

- 1) The paper must use some form of POSG abstraction with the goal of enhancing some performance metric. Hence, we do not include abstractions aimed at improving interpretability.
- 2) The employed abstraction must not be learning-based. Each abstraction method should be employable on the spot for every decision point requiring only a moderate level of computational overhead. In particular, this excludes deep learning-based abstractions.
- 3) The paper must apply its abstraction to a sequential decision-making task. This may include multi-agent settings, partially observable problems, or problems with stochasticity. However, we exclude perfect information, single-agent, and deterministic problems as we do not consider them sequential since one could preplan an optimal action sequence from the initial state.

These criteria include papers that use a learning-based inference method on top of a non-learning-based abstraction (e.g. think of reinforcement learning algorithms that are employed on a simplified MDP) since we are only interested in the abstraction. Following the main motivation of this survey, for Chapter V where we show how abstractions can be utilized and Table I where we give a paper overview, we only considered algorithms whose entire pipeline i.e. the abstraction and the usage of the abstraction is not learning based (e.g. tree searches on top of a non-learned abstraction).

For this survey, we considered all papers that satisfy the above-mentioned criteria and are linked through citations to the top 500 results on Google Scholar obtained on the 11th of September 2024 using the two keywords *abstractions* and *decision-making*. In addition, we also included papers we had prior knowledge about.

IV. ABSTRACTION CONSTRUCTION METHODS

In this section, we will review different concepts for constructing an abstraction. The first concepts that we will present are

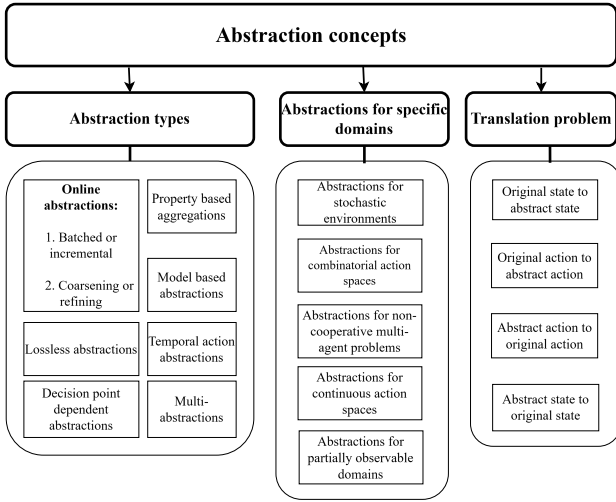


Fig. 1: Abstraction concepts overview.

different types of abstractions (e.g. lossless or model-based abstractions). Afterwards, we will discuss abstraction techniques for specific, challenging domains such as continuous action spaces and lastly, discuss an often overlooked component of abstractions which is its state and action translation. Therefore, a single abstraction method can rarely be described by a single concept, but rather a set of concepts. An overview of the concepts of this section is given in Figure 1.

A. Online abstraction construction

Some abstractions may be constructed in parallel to the search or optimizing technique they are being employed in. One differentiates between two approaches that come with a different tradeoff between precision and computational cost: batched and incremental construction techniques.

1) *Batched construction*: Batched abstraction constructions pause the optimization or search procedure in regular intervals to update the current abstraction with the latest search information. A prototypical example of this is given by Jiang et al. [71] who regularly pause *Upper Confidence Trees* [26] (UCT) to update their empirical MDP model whose refined version is then used to further enhance UCT. This batch style of updating the abstraction is quite common in the literature [4], [71], [139], [140].

2) *Incremental construction*: Incremental abstraction construction constantly tries to update the abstraction to avoid information lagging too far behind of what is obtained during the search. However, these methods are often much more computationally expensive than their batch counterparts. A prime example of this approach is *On-the-Go Abstractions* (OGA) by Anand et al. [5] who recompute the abstraction of an UCT tree at nodes when they are visited and their visit count is divisible by some predefined number to ensure that the abstraction does not lack behind the latest information contained in frequently visited nodes. This incremental construction style is mostly used in sample-based searches [5], [68], [69], [115] but it also appears in α - β -search-based (a

modification to *Minimax* [40]) approaches [56].

Online abstractions can also be differentiated by whether the abstraction’s granularity increases (refining abstraction) or decreases (coarsening abstraction) during the search/optimization. The granularity of an abstraction is a measure of its complexity compared to the original model. The finer the granularity of an abstraction, the higher its complexity (e.g. the number of abstract states or actions) relative to the original model. The finest abstraction is having no abstraction at all and the coarsest abstraction is a trivial model where all states and actions are in one group.

1) *Refining abstractions*: A refining abstraction starts with all states or actions being in the same (or one of a few) group(s) and does the construction by splitting these groups into smaller ones. One such approach has been demonstrated by Hostetler et al. [68], [69] who proposed an abstraction that initially puts all possible successors of a node-action pair of a search tree in the same group. The abstraction is then iteratively refined by selecting one abstract node and splitting it into two abstract nodes. Sokota et al. refine an abstraction during a tree search by letting a parameter controlling the abstraction’s coarseness converge to zero [115].

2) *Coarsening abstractions*: Contrary to refining abstractions, one starts with the original model (or a high-complexity abstraction) and does the construction by aggregating individual states or actions. An example of this is *AS-UCT* by Jiang et al. [71] based on the concept of approximate homomorphisms [99] where each node of a search tree is initially considered unabstrated. Starting from the bottom layer of a UCT tree, up to the root, they group nodes in the same layer if, for each action, their transition probabilities to the (abstracted) nodes of the previous layer and the associated rewards are within predetermined thresholds. Extensions of this exploit more symmetries to group more nodes [4], [5].

An interesting approach that combines both a coarsening and refining abstraction is *Elastic MCTS* by Xu et al. [139] that first builds a coarsening abstraction during Monte Carlo Tree Search (MCTS) [26] which is abandoned once an iteration threshold has been passed which can be viewed as a refinement.

In general, online abstractions directly depend on the current state of the search or optimization algorithm. In particular, state groupings may directly depend on parent nodes in search trees [28], [104], currently expanded children nodes [4], [5], [71], [106], [139], [140], or sibling nodes [115]. Furthermore, it is also possible for an abstraction to directly depend on current search statistics. An example of this is given by Bruno Bouzy [22] who grouped actions based on their Monte-Carlo search statistics alone. He simply tracked the current mean outcome of each action for a state and split the actions into two groups called *GOOD* and *BAD* where *GOOD* contains the top half of actions sorted by their mean and *BAD* the lower half.

B. Lossless abstractions

Usually, abstractions reduce the problem’s complexity at the cost of losing information which means that an optimal strategy in the abstraction may not directly correspond to an optimal strategy in the original problem, or even worse the optimal strategy cannot even be represented in the abstraction. Such abstractions are *lossy*. Their counterparts are *lossless* abstractions which can be constructed, for example by grouping symmetric states in board games (e.g. there are eight symmetries in Othello), or grouping suit-isomorphic hands in Poker [17], [54], [74], [147]. The complexity reduction of these lossless abstractions, however, is rather limited.

Only in rare cases with strong assumptions can lossless abstractions substantially reduce the problem’s complexity such as demonstrated by Cano et al. [27] who considered non-markovian tasks and grouped histories based on the value of a statistic of this history. They assumed the existence of a statistic such that the reward and transition functions only depend on the value of the statistic.

C. Decision point dependent abstractions

During a sequential decision-making process, the optimal action to take depends on the current state and so may an abstraction. Therefore it can be beneficial to construct abstractions dependent on the decision point.

One general approach is to increase the abstraction granularity locally around the currently occupied state. For example, while Sven Behnke [14] used a multiresolution grid to abstract a region of $\mathbb{R}^n$ with the highest resolution near a robot’s current location, Nashed et al. [91] refined all abstract states for which at least one of their constituents has been visited in the current episode.

For partially observable domains, the abstraction might be dependent on the current belief state. An example of such abstractions is those using the *free space assumption* that is explained in Subsection IV-L where the abstraction, in essence, is constructed by guessing the hidden state.

D. Property-based aggregation

An abstraction of the state/action space may be accomplished by aggregating or grouping states/actions into an abstract state/action, based on some properties of these states/actions. Properties are mappings from states or actions to $\mathbb{R}^k$ that can in principle be computed without accessing the model the state or action is part of.

One simple approach could be to consider states that can be characterized as a list of properties (i.e. a list of units and their attributes for states in strategy games) and group states whose property lists coincide when discarding a handpicked or algorithmically generated set of properties thereby ignoring irrelevant details. This has been done for strategy games [45], board games [105], and propositional

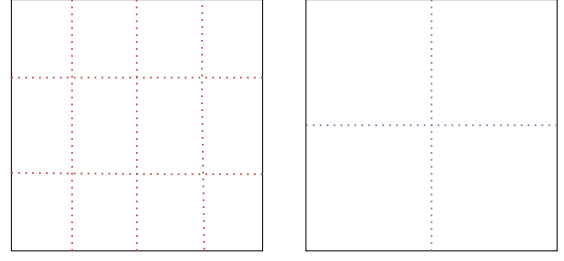


Fig. 2: Two possible tilings of the state space $S = [0, 1]^2 \subseteq \mathbb{R}^2$. The abstracted space on the left is $S_{abs,left} = \{[\frac{k}{4}, \frac{k+1}{4}) \times [\frac{m}{3}, \frac{m+1}{3}) \mid 0 \leq k \leq 3, 0 \leq m \leq 2\}$ and the abstracted space on the right is $S_{abs,right} = \{[\frac{k}{2}, \frac{k+1}{2}) \times [\frac{m}{2}, \frac{m+1}{2}) \mid 0 \leq k \leq 1, 0 \leq m \leq 1\}$.

logic based MDPs [39]. Analogously this may also be done for actions. For example, Richard Lorentz [84] grouped actions in the game Amazons that consist of choosing an Amazon, a tile to move, and another tile to throw a spear at by discarding the spear throw position.

Aggregation becomes simpler when the relevant property is only 1-dimensional (i.e. a real number). For states in Poker games such a property could simply be the average hand value of all possible completions [110] or for states that are histories of atomic observations it could be some statistic about the sequence of observations [27]. Groupings may then be created solely based on the property, for example, by grouping all states with the same property value [27] or by dividing the interval between the value’s minimum and maximum value into equally sized subintervals [72]. If one can efficiently determine the quantile a property is in or how many states map to each property value (which arguably is not the case for the following citations), then one could also create groupings by running k -means clustering [53] or by putting all states in the same quantile [72], [109], [147], [148] into one group.

When the state/action property is multidimensional (i.e. in $\mathbb{R}^k, k > 1$), for example, in factored MDPs, this property could be the state or action itself, then some of the above-mentioned techniques for one dimension can be generalized. For instance, the above-mentioned techniques can be applied independently to each dimension [91]. An example of this is *tiling* where one discretizes $\mathbb{R}^n$ into a grid of equally sized hypercubes [122], [124] (visualized in Figure 2). Alternatively, when not treating the dimensions as independent, one could divide the now multi-dimensional property space into disjoint, not necessarily evenly sized regions (possibly by an expert) and group states based on whether their property vectors lie in the same region [6], [7], [122], [124].

Aggregation may also be nested where a state/action may be grouped by one set of properties first and then inside these coarse groups, grouped again by a different set of properties [72].

E. Model-based abstractions

Instead of building an abstraction by grouping states or actions based on properties, one can also construct abstractions based on an analysis of the model's transition and reward function if one can access them or at least sufficiently well approximate them.

This concept can be applied by grouping states or actions that are strategically similar or symmetric. A prime example of this is AS-UCT as already described in Subsection IV-A or one of its successors that additionally exploit symmetries like *ASAM-UCT* that group states if their actions can be mapped to each other or *ASAP-UCT* that additionally decouples action and state abstractions. In general, using approximate homomorphism-based approaches has seen widespread use [4], [5], [71], [106], [139], [140].

A more aggressive model-based abstraction approach applicable to round-based games is to cut the game tree off after a fixed number of rounds. For example, Zinkevich et al. [147] limit the number of bets per betting round. In addition to limiting the number of bets, Billings et al. [17] truncate the game tree before the last betting round and use a Monte-Carlo evaluation as the payoffs for the new terminal states. Similarly, for online settings, one may restrict oneself to an abstract local MDP for planning consisting only of states in a neighborhood of the current state [128].

For imperfect information games, the current information set is directly dependent on all previous atomic observations and actions taken. However, for many problems, a good heuristic is to assume that only the most recent atomic observations or actions are the most relevant. *Imperfect-recall* abstractions may discard previous atomic observations or actions when creating an abstraction. For example, in Poker, an imperfect-recall-based abstraction is to group information sets based only on previous actions and the current hand, thus deliberately forgetting previous hands and the groups they were in [73], [137].

A hybrid-like approach has been used by Dearden and Boutilier [21], [39] who exploit the problem model to find a set of state components for state spaces that are subsets of $\{0, 1\}^n$ that significantly influence the reward function which are then used for a property-based aggregation.

F. Temporal action abstractions

One class of action abstraction is *temporal actions* (or synonymously *macro actions*) where one temporal action corresponds to a (possibly variable size) sequence of original actions generated by some predefined behavioral policy.

A classic example of temporal actions is repeated actions that are described as tuples of an original action a and an integer n whose application corresponds to applying a , n consecutive times. These types of actions can be especially beneficial

when a single time step barely changes the environment and thus yields little to no feedback for an optimizer as is the case for Atari games where one usually repeats the last chosen action for a fixed number of frames which is called *frame skipping* [24], [76].

Repeated actions are static in that the action sequence is predetermined and will not change no matter the incoming observations. Adaptive temporal actions are common in RTS games where one (temporarily) delegates the decision-making to a scripted agent, for example, until a terminal state [10]–[12], [31], [75], [78], [85], [87], [88], [134] or until some condition defined by the scripted agent itself is met [12].

Aside from manually defined temporal actions, it is also possible to find a temporal action by randomly sampling trajectories from a given state and choosing those as temporal actions whose final states fulfill a (handpicked) subgoal predicate (e.g. reaching a certain point on a map) [50] or by following the actions of a search algorithm conditioned on the goal predicate of the temporal action [7], [64].

G. Multi-abstractions

Some abstractions can be decomposed into multiple abstractions. We call these *multi-abstractions*. A classic example of multi-abstractions is to combine abstraction methods that are independent of each other, for example, by combining state and action abstractions as has been the case for Poker variants [17], [25], [55], [63], [109], [136] or strategy games [45].

Another multi-abstraction approach is to simply create n abstractions all of which could in principle be used as a standalone. For example, one could randomly and independently create n abstractions, solve the problem on each abstraction, and pick the action with the highest value averaged on all abstractions as was the case for *FF-Hindsight* [143] or handpick a number of independent abstractions with different levels of granularity and decide during the decision-making process which abstraction to use [126].

One family of multi-abstractions is hierarchical abstractions in which abstractions form a hierarchy. Each layer's abstract problem is to solve a task given by the previous layer which is itself accomplished by assigning tasks to the next layer similar to the chain of command concept in militaries as noted by Kenrick Mock [86]. Usually, only the bottom layer shares actions with the original model. A prime example of a two-layer hierarchy is the usage of temporal actions (see Subsection IV-F) where the highest layer's action space consists only of temporal actions (perhaps in addition to a state abstraction) while the bottom layer is tasked with executing a given temporal action on the original problem [7], [100]. The usage of hierarchical abstractions ranges from RTS games [117] to dynamic constraint satisfaction problems [49] and they may deviate from the strict chain of command

structure and allow, as an abstract action, to deny the current task (e.g. if it has become infeasible) [49].

H. Abstractions for stochastic environments

In non-deterministic (or stochastic) environments, a state-action pair may branch off into several successor states. In highly stochastic environments this might cause search techniques such as MCTS or *Iterative Deepening* [77] to degrade to a shallow search. In this subsection, we want to highlight abstraction techniques to explicitly deal with stochastic environments.

One family of approaches called *determinization* transforms a stochastic problem model into a deterministic one. This can be done by picking a single successor for each state-action pair for example by random sampling [143], picking the one with the highest probability [142], or replacing each original action with one abstract action for each possible successor [142].

Instead of fully determinizing a model, one could reduce the stochastic branching factor by allowing only a subset of outcomes, for example, keeping only the two most probable outcomes [103]. A more advanced example is demonstrated by Sokota et al. [115] who proposed *Abstraction Refinement* as an alternative to *Progressive Widening* [38]. Whenever a successor state of a state-action pair has to be sampled, *Abstraction Refinement* checks whether that sample is in the same abstract state (determined by a similarity measure) as another, already sampled successor of the same state-action pair. If so, the latest sample is discarded and the previous sample is used instead.

Furthermore, the abstraction can be decided on a per state-action pair basis as shown by Saisubramanian and Zilbertsein [103] who either fully determinize a state-action pair or do not change its outcomes at all depending on their *reduction impact*, a heuristic measuring the risk-negligence of the determinized model (e.g. ignoring potentially catastrophic outcomes has a high reduction impact).

I. Abstractions for combinatorial action spaces

One successful attempt to reduce the complexity of problems with combinatorial action spaces such as unit-based games is to abstract the action spaces with so-called *portfolios*. Formally, portfolios are sets of *scripts* which themselves are functions mapping from states to actions or partial actions. Intuitively, a portfolio represents a strategy space and scripts implement a specific strategy.

A straightforward portfolio usage is to replace the original action space with an action space whose only available actions are choosing a script from a portfolio. This is equivalent to filtering the original action space at each state to those actions that are the output of at least one script. An example where a portfolio is used to replace the action space is the family of so-called *Rush-scripts* for Starcraft that are able to play

an entire Starcraft game by constantly building one type of unit and sending it to attack once a unit threshold has been reached in the hopes of overwhelming the opponent before they can build defenses [12], [93]. Portfolios that contain scripts capable of playing an entire RTS game have also been developed for μ -RTS [85], [92], [111], [118] or for even more simplified RTS models [101].

Scripts may also be parametrized. Choosing an action for a state then corresponds to choosing a script from a portfolio as well as its parameter values. Of course, any parametrized script can be viewed as a set of non-parametrized scripts, however, taking the parametrized viewpoint allows one to have a $(n + 1)$ -dimensional abstract action space instead of a 1-dimensional one (i.e. one dimension for the script and one for each of the $n \in \mathbb{N}^+$ parameters). The above-mentioned Rush-scripts can be viewed as one script that is parametrized by a unit type and the number of units needed for an attack. This viewpoint has been taken by Bariga et al. [11], [12].

In case the action space can be described as the cross-product of component action spaces (e.g. action assignment for each unit in a strategy game), one can use a single or multiple portfolios to replace the component spaces. Such portfolios have been developed for strategy games such as Stratega and Tribes where scripts suggest an action for an individual unit or building [44], [97], for Prismata where each script suggests an action for one action phase [32] or for RTS games like Starcraft where a particularly popular unit script is called *No-OverKill-Attack-Value* [31], [33], [78], [134] that attacks any opponent unit when within range or moves towards the enemy unit with the highest damage-health ratio. If that unit is already assigned lethal damage this round, the next highest-scored unit is targeted.

Still, even with a portfolio replacing each component action space, the action space may still suffer a combinatorial blowup. One possibility is to first group component action spaces and only allow the assignment of a single script to an entire group. This has been implemented by Levi Lelis [78] for SparCraft who assigned the same script to all units that had the same attack range, attack damage, and similar health values. Assigning the same action or script to entire groups of units has also been done by Justesen et al. [75] and Ontañón and Buro [92]

Up until now, we have only considered symmetrical techniques that assign the same portfolio to each component action or action group. However, not every component action might be equally important, it could be beneficial to assign a bigger, more expressive portfolio to high-impact components. For SparCraft, Moraes and Lelis [88], [90] propose to select a set of high-impact units (e.g. ranked by attack value) to allow these to have access to the full action space while other units have to use a portfolio. Asymmetric portfolios have also been employed by using different portfolios for different unit types

[89], [90] or for games where the action components share no legal action such as Prismata [32] with its different phases.

J. Abstractions for continuous action spaces

Similarly to the problem with highly stochastic problems, in the case of continuous action spaces or discrete ones with a large number of available actions, any search algorithm also likely degrades to a shallow search if this problem is not addressed properly. One fix is to pick a small number of representative actions from the original action space similar to how outcomes from state-action pairs in determinization from Subsection IV-H were picked. For continuous action spaces of the form $[a, b]^m$ one may pick equally spaced representatives (e.g. for $K \in \mathbb{N} \setminus \{ \frac{2aj}{K-1} - b \mid 0 \leq j \leq K-1 \}^m$ [127]) or for Poker allow only certain multiples of the current pot as the only available bet sizes [55].

Instead of handpicking a number of representatives, they may also be automatically determined during the search itself. For example, each node may start with no available actions, and actions are added progressively at fixed node visit counts. The added representants may be obtained by randomly sampling an action [135], by sampling multiple actions and choosing one that best balances novelty and proximity to an action with a high Q value [36], [37], or by including the mean of the two best-performing actions [16].

An entirely different approach is to use discrete abstract actions that define how to modify the last chosen action which can be useful for domains where the optimal policy's action has few drastic changes from one time step to the next. For example, Pakis and Logoudakis pick an initial continuous action from a one-dimensional continuous action space and then discretize it either by a 2-dimensional discrete action space that includes only whether to increase or decrease the last action taken [96] or by n sequential increase-or-decrease decisions to choose a continuous action with binary search [95] as visualized in Figure 3.

K. Abstractions for non-cooperative multi-agent problems

One approach to dealing with non-cooperative multi-agent settings is to abstract the problem into a single-agent problem by opponent modeling. For example, Xu et al. [139] assume a static, deterministic rule-based agent during MCTS as the opponent for a two-player game. This effectively doubles the search depth at the cost of increasing one's own exploitability since most opponent's actions are not taken into account. Xu et al. applied this technique to a fully observable setting, however, opponent modeling is also applicable to imperfect information games [34].

Usually, if the opponent is not statically modeled as described above, a symmetric abstraction is used, which abstracts states and actions associated with one player to be the same for both players. However, one might also consider using an asymmetric abstraction, for example, to reduce

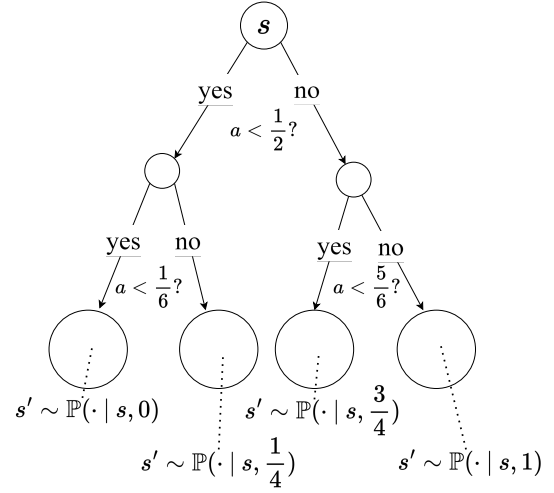


Fig. 3: Visualization of reducing the continuous action space $A = [0, 1] \subseteq \mathbb{R}$ to a sequence of binary decisions. In this case, there are $n = 2$ decisions.

exploitability by using a finer abstraction for the opponent. Experiments on asymmetric abstractions have been conducted by Bard et al. [9] for Texas Hold'em who used an asymmetric state abstraction.

L. Abstractions for partially observable domains

One way to employ abstractions for partially observable domains is to construct the abstraction in such a fashion that the resulting abstract problem is fully observable. This can be accomplished by simply guessing the hidden state and planning based on this guess. The plan is then executed and continuously updated given new observations. This guess may be optimistic, for example, in path planning for unknown environments one may use the *free space assumption* which assumes that any unexplored tile does not contain an obstacle. Planning is then conducted under the free space assumption and whenever an obstacle is hit during the plan execution, the plan is updated [120], [121]. The guess could also be pessimistic to account for all possibilities by assuming the hidden state is the worst possible one for the decision maker. For dynamic path planning this could mean assuming an obstacle at each unknown position [130] or assuming obstacles simultaneously occupy every position they could possibly be at given their noisy position [14].

Instead of assuming a concrete hidden state, one may randomly sample multiple hidden states, solve each perfect information instance, and derive a decision for the original game from the perfect information results as done by Ginsberg [57] who sampled the opponent's hands in Bridge to suggest moves in the card play phase. Similar approaches have been applied to Doppelkopf [41] and Hearthstone [42].

Another alternative to assuming a concrete hidden state is a

method similar to transforming a POMDP to a belief-state MDP. The idea is to use abstract states that are supersets of the current information set as these might be easier to represent than the information set itself. For example, Bolognesi and Ciancarini [19], [34] represented a Kriegspiel position by the known position of the allied pieces, meta-information, and for each tile-piece pair whether the tile could possibly contain the piece. Of course, to use these abstract states, one needs to define an appropriate transition function and action space for the abstraction too. Bolognesi and Ciancarini chose as abstract actions, all original actions applicable in at least one of the constituent states [19], [34] and used either a deterministic, handcrafted, heuristics-based transition function which implicitly modeled the opponent [34] or a non-deterministic one where each action may branch off into several successor abstract states thereby accounting for the opponent's behavior [19].

M. Abstraction translation methods

Ultimately, abstractions are only a tool for computing actions in the original model. Therefore, one needs to be able to translate states or actions from the original model to the abstraction and vice versa.

1) *Original state to abstract state:* The translation problem from an original state to an abstract state does not receive much attention in the literature as the translation usually directly follows from the abstraction definition. For example, for property-based state aggregations (see Section IV-D) one essentially only needs to identify the state's property vector, or for imperfect recall abstractions one simply ignores past atomic observations and actions and considers only the latest atomic observation.

2) *Original action to abstract action:* In the case of imperfect information games with perfect recall, translating the current original observation which is the history of atomic observations and actions, the state translation problem contains the action translation problem as a subproblem, therefore one sometimes needs to translate an original action into an abstract action. As with the state translation, this problem can be trivial when the original action is part of the abstract action space. Otherwise, it is common to map the original action to the most similar abstract action. For example, for no-limit Texas Hold'em Poker, Gilpin et al. [55] suggested considering the two abstract actions d_1, d_2 surrounding an original action c (actions are bet sizes), i.e. $d_1 < c < d_2$ and pick d_1 if $c/d_1 < d_2/c$ and d_2 otherwise. Schnizlein et al. [108], [109] argued that such a deterministic mapping causes exploitability and suggested *Soft Translation* that non-deterministically maps original actions by sampling them based on similarity to the abstract actions.

3) *Abstract action to original action:* Translating an abstract action into an original action is trivial when the abstract action directly corresponds to an original action. For example, in no-limit Poker variants, the action abstraction is often a subset of allowed bet sizes [25], [55], [63], [108], [109], [136], for Portfolio methods, the chosen script by construction outputs an original action.

It is also possible for some abstractions to have been constructed with a specific (not necessarily trivial) translation in mind. For example, Chung et al. [29] partitioned an RTS map into tiles and one abstract action is moving each unit to a nearby tile. This was translated to a sequence of original actions by running an A* search [62] for each unit to find paths to the selected tiles. This technique is common for RTS games [8], [131].

Some abstractions are deliberately designed on a high level to leave more freedom (and thereby room for improvement) to the translation task which itself can then become an optimization problem of its own. For example, Stanescu et al. [117] abstracted an RTS into a plan selection problem where a plan is composed of pairs of tasks where a task is a selection of a high-level objective (e.g. attacking or defending a base) and a group of units to accomplish it with for both players. The translation in this case to unit-level actions was done with α - β search or Portfolio Greedy Search (PGS) [31].

4) *Abstract state to original state:* A translation from an abstract state to an original state is rarely of interest as one only needs the abstraction to determine an original action.

V. APPLICATIONS OF ABSTRACTIONS

This section is structured as follows. First, we will present a formalism to describe methods that use abstractions. Afterward, we will present different paradigms to incorporate an abstraction into a non-learning-based decision-making algorithm and describe them using the previously introduced formalism.

To our knowledge, there is no blueprint/pseudocode for the general application of an abstraction to a search or optimization technique. We believe that finding such a framework would not be of much use as it would be too generic. Instead, we propose to categorize the application of abstractions by their traversal pattern of a specific graph type, which we call *Abstraction Mapping Graph (AMG)*. Before, introducing AMGs we need to define a *state-transition graph*.

Definition V.1. Let $M = (n, S, \mu_0, A, \mathcal{O}, \mathbb{P}, R)$ be a POSG. We define the *state-transition graph* of M as a directed graph $G_M = (S, E)$ with $(s_1, s_2) \in E \subseteq S \times S$ iff there is an $a \in A$ such that $\mathbb{P}(s_2 | s_1, a) \neq 0$.

On a high level, an AMG is a graph with two layers, one is the state-transition graph of the original model and the other one is the state-transition graph of the abstract model. Additionally, the AMG has edges connecting these two layers to capture how the abstraction relates to the original model. More formally:

Definition V.2. Let M, M_{abs} be two POSGs and $G_M = (S, E), G_{M_{abs}} = (S_{abs}, E_{abs})$ be their state-transition graphs with $S \cap S_{abs} = \emptyset$. An *AMG* of M, M_{abs} is any graph G of the form $G = (S \cup S_{abs}, E \cup E_{abs} \cup E^*)$ with $E^* \subseteq S \times S_{abs} \cup S_{abs} \times S, (v, w) \in E^* \implies (w, v) \in E^*$.

AMGs can be used to describe the application of abstractions as follows. Given an observation $o_i, o \in \mathcal{O}$ for the i -th

player in a POSG M , the decision-making process of any abstraction-using algorithm can be viewed as alternately constructing an abstraction of M , where we denote the k -th ($k \leq l \in \mathbb{N}$) thus obtained abstraction by $M_{abs,k}$ and then implicitly traversing, without modifying, an AMG of M and $M_{abs,k}$. Hence, one first needs to identify M and the $M_{abs,k}$ and then define an AMG G_k for each pair $(M, M_{abs,k})$ and denote how G_k is traversed. In the following, if the AMG G_k is clear from context, E_k^* ($E^* := E_1^*$) denotes the edges from G_k connecting the two state-transition graphs of M and $M_{abs,k}$. Furthermore, $E_{abs,k}$ denotes the edges from $G_{M_{abs,k}}$, E denotes the edges from G_M and we use the abbreviation $E_{abs} := E_{abs,1}$.

Next, we give some examples of the $(M_{abs,k}, G_k)$ sequences. In AS-UCT [71] $M_{abs,k}$ is an abstraction of the tree-POSG rooted at the current state and $M_{abs,k}$ transitions to $M_{abs,k+1}$ in regular UCT iterations intervals. E_k^* might be defined as connecting each abstract state with all its constituent states. In FF-Hindsight [143] $M_{abs,1}, \dots, M_{abs,l}$ are the random determinizations of the original model that are successively constructed and solved. Since the states of G_M and $G_{M_{abs,k}}$ coincide, E_k^* could be defined as connecting the state pairs. Examples of methods using static abstraction where only $M_{abs,1}$ exists are those that group symmetric states in board games followed by some tree search on $M_{abs,1}$ for decision-making. Again, E^* could simply be defined by connecting each abstract state with all original states it groups.

There is rarely a unique way of using an AMG to describe the application of an abstraction. After all, the definition of an AMG is very unrestrictive. This is not an issue but rather indented as AMGs should be used to look at the usage of an abstraction from different viewpoints. For example, using MCTS move groups (see Subsection V-B) with $l = 1$ can be viewed as a fully abstract search where one only operates in the abstract model (see Subsection V-A) where each original action is replaced by an abstract action, state, action sequence. Alternatively, to emphasize that the abstraction is only designed to enhance the node selection phase, one may view the search as traversing the original model only except during the tree policy where no edges from E are traversed, instead, one takes an edge in E^* to the corresponding abstract state, then two edges in E_{abs} (corresponding to choosing a move group and an action from the chosen group) and then traversing back to the original state with another edge in E^* which is illustrated Figure. 4.

A. Fully abstracted optimization and search

The opposite of not using an abstraction is to fully abandon the original model. In terms of the AMG this means that $l = 1$ and directly traversing from the start state to an abstract state using E^* , doing all the computation in the abstract model, i.e. only using edges in E_{abs} and only at the end, converting the computed abstract action to an original action. This has been a popular strategy in the context of RTS games

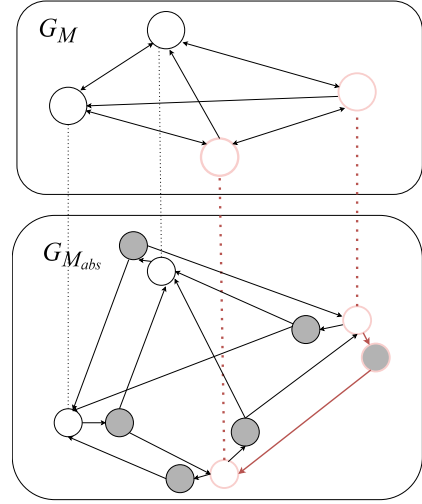


Fig. 4: AMG traversal illustration for a move groups abstractions. The color grey indicates move-group nodes, dotted lines are edges from E^* , and the traversed AMG path for selecting an original action is marked red.

where the abstracted model is mostly handcrafted using expert knowledge. These models themselves may even vary in their level of abstraction. They could still resemble the original game where one abstract action corresponds to a sequence of original actions [8], [29], [131] or be a highly abstracted 1-step game where for example the only actions are which strategy is to be played [29].

Usually, one can employ a standard search or optimization algorithms for computing an abstract action on the abstraction. For example, Monte-Carlo planning is the underlying abstraction solver for *MCPlan* [29], α - β search is one of the solvers for *Hierarchical Portfolio Search* (HPS) [32], an evolutionary algorithm is used in *Portfolio Rolling Horizon Evolutionary Algorithm* (PRHEA) [44], a greedy hill-climb search is used for PGS, or *MaxiMax* was used for *Darkboard*, a Kriegspiel agent [34].

B. MCTS node selection phase improvement

Given a state abstraction, one can simply use it to get a better estimate of the UCB value in UCT. During UCT, the UCB value of a node n is then determined by the aggregated visit and average value of all nodes that are in the same group (or abstract node) as n [4], [69]–[71], [139], [140]. This allows information in the search tree to flow between nodes of different branches. In particular, this can allow one to have a good value estimate of nodes with a low visit count. In terms of AMG traversal, this technique corresponds to traversing only edges in E_k^* or E .

One approach to using action abstractions are so-called *move groups in MCTS*. Here, the node selection in UCT is replaced by a two-stage system. First, the actions are grouped and one group is chosen using the UCT selection formula, then in the

second stage, one action from the chosen group is selected, again, using the UCT selection formula. This approach has been applied to toy problems [48] and board games [28], [104] and generalized by Xu et al. [139] to an n -stage system for Stratega with their method named $MCTS_u$ where first a random move order for the units is sampled and then each action is broken down into sequentially selecting an individual unit action.

Action abstractions, in particular scripts, can also be used to improve the order of expansion at non-fully expanded nodes. For example, Churchill and Buro [31], [33] employ scripted move ordering, where the first moves to expand are those suggested by scripts. Similarly, Yang and Ontañón [141] use a single script to expand the first node only in *naïveMCTS* (MCTS with a modified sampling strategy based on Combinatorial Multi-Armed Bandit problems [93]) in a method they call *First-Choice Guided Naïve Sampling*. In an extension, called k -GNS, they sample uniformly from a portfolio to expand the first node. From thereon, randomly, a random script from this portfolio is sampled whenever a new node needs to be expanded, in that case overriding the naïve sampling procedure. In terms of AMG (assuming the portfolio is fixed, i.e. $l = 1$), the abstraction-enhanced expansion can be described by taking an edge in E^* , then one in E_{abs} (i.e. choosing a script) followed by one in E^* to get back to the original state.

C. Improved α - β search pruning

The standard branch pruning in α - β search can be complemented using state abstractions. For example, Matthew Ginsberg [56] builds a state abstraction during search such that all states in one abstract state have approximately the same minimax value. Before any node is recursively expanded, it is checked whether another state with the same abstract state has already been expanded whose value is then immediately returned. Like the previously mentioned UCT improvement (see Subsection V-B), in terms of AMG traversal, this technique corresponds to traversing only edges in E_k^* or E .

D. State evaluation improvement

Some optimization or search algorithms like MCTS or α - β search require the evaluation states which can be improved using abstractions. One technique is to use an action abstraction to speed up the rollout phase of an MCTS. Usually, one generates all available actions per state and then uniformly samples one to apply to the current rollout state. However, this can be quite time-consuming for games with enormous action spaces. Instead, one could use move groups (see previous Subsection V-B) to first sample an action group and then sample an original action inside this group. This idea has been applied by Richard Lorentz [84] to the game Amazons where one state can have up to 1000 legal moves.

As mentioned above, the MCTS rollout phase usually samples uniformly from all available actions until a terminal

state or to a certain depth at which point an heuristic evaluation function is used. To obtain a more precise value estimate, one can use an action abstraction to guide the rollout. This idea can be implemented through portfolio methods. Instead of evaluating a state s one could use a scripted payout for a fixed duration or until a terminal s' and return the evaluation of s' for the evaluation of s . This technique has been extensively used to evaluate Starcraft (or SparCraft) game states [10]–[12], [31], [75], [78], [85], [87], [88], [134].

VI. RESEARCH DIRECTIONS

In this last section, we will discuss promising avenues for future research that we identified by conducting this survey.

1) *Benchmarking*: Abstraction techniques in general have to be more comparable. It is only rarely the case that papers test their algorithm on more than 3-4 test problems even though their technique could be applied more generally. This has the effect that techniques that can be applied to the same domains have not been compared. For example, there is no reason why many of the portfolio techniques used for Starcraft cannot be applied to Stratega, Hero Academy, or Tribes and vice versa. Therefore, it should be a future endeavor to create benchmarks for specific types of POSGs like the International Probabilistic Planning Conference (IPPC) [20] has done for perfect information, stochastic MDPs but for example for POMDPs, strategy games and card games and establish the usage of these.

2) *Combining methods*: Given the large number of abstraction techniques, it will be interesting to combine abstractions that are independent of each other. This does not necessarily have to involve combining only action and state abstractions. In particular, determinization approaches or other abstractions that try to reduce the stochastic branching factor could very well be combined with abstractions aiming at reducing the action branching factor.

Furthermore, it could be interesting to integrate some of the here-mentioned abstractions, wherever applicable, with other solvers that on their own do not necessarily need an abstraction. In particular, some deep learning-based agents which use tree search for inference could be further enhanced with abstractions such as the guided MCTS in *AlphaZero* [112].

3) *Dynamic abstractions*: Usually, there are several abstraction techniques for a single problem. This is usually not a problem as one can empirically determine the best abstraction to use. However, there might not always be a single best abstraction for a problem but only for specific decision points. It is an understudied problem of dynamically choosing which abstraction out of a set of possible abstractions to use. Perhaps at some states, the abstraction built by Forward Search Sparse Sampling [68], [69] (FSSS) is stronger than that of ASAP-UCT [4] while in others ASAP-UCT is stronger. Furthermore, very little research has been conducted on abstractions that are heavily dependent on the current decision point like increasing the granularity around of the abstraction around the current state.

4) *Investigating implicit design choices:* Some algorithmic choices for abstraction usage have been established by early papers without providing any reasoning for these and then silently adopted by subsequent works. These should be thoroughly investigated in future works. In particular, the usages of state abstractions for improving MCTS (see Subsection V-B) randomly select a state within the chosen abstract state. What would happen if this is done through another UCB procedure or greedily? Also, the exploration term always uses the aggregated visits but is this truly necessary?

5) *Abstraction quality impact on abstraction usage methods:* Only for some abstraction application methods has the impact of the choice of abstraction been studied. In particular, the incorporation of abstractions of varying levels of quality into UCT to improve the UCB value could be thoroughly empirically investigated. In this case, an abstraction is only used to enhance the UCB value. One could study how performance depends on the abstraction quality which could be precisely tuned by using an oracle to determine the true Q value of newly initialized nodes and use a noisy version thereof for UCB.

By categorizing and discussing various abstraction techniques and their application in search algorithms, this survey aims to support practitioners and researchers in selecting and integrating suitable abstractions to improve computational efficiency and performance in complex decision-making tasks. This structured overview underscores the critical role of abstractions in advancing sequential decision strategies across diverse applications. We hope that it will prove useful in following up with the suggested research directions.

REFERENCES

- [1] Midjiwan AB. The Battle of Polytopia, 2016.
- [2] Activision Blizzard. Hearthstone. <https://hearthstone.blizzard.com/en-us> Accessed: 2024-11-04.
- [3] Phillip Alder and Albert H. Morehead. Bridge. *Encyclopaedia Britannica*, 2024. <https://www.britannica.com/topic/bridge-card-game> (Accessed: 25.9.2024).
- [4] Ankit Anand, Aditya Grover, Mausam, and Parag Singla. ASAP-UCT: Abstraction of State-Action Pairs in UCT. In Qiang Yang and Michael J. Wooldridge, editors, *Proceedings of the Twenty-Fourth International Joint Conference on Artificial Intelligence, IJCAI 2015, Buenos Aires, Argentina, July 25-31, 2015*, pages 1509–1515, Washington, DC, 2015. AAAI Press.
- [5] Ankit Anand, Ritesh Noothigattu, Mausam, and Parag Singla. OGA-UCT: on-the-go abstractions in UCT. In *Proceedings of the Twenty-Sixth International Conference on Automated Planning and Scheduling, ICAPS'16*, page 29–37, Washington, DC, 2016. AAAI Press.
- [6] Dilip Arumugam and Satinder Singh. Planning to the Information Horizon of BAMDPs via Epistemic State Abstraction. In Sanmi Koyejo, S. Mohamed, A. Agarwal, Danielle Belgrave, K. Cho, and A. Oh, editors, *Advances in Neural Information Processing Systems 35: Annual Conference on Neural Information Processing Systems 2022, NeurIPS 2022, New Orleans, LA, USA, November 28 - December 9, 2022*, 2022.
- [7] Aijun Bai, Siddharth Srivastava, and Stuart Russell. Markovian State and Action Abstractions for MDPs via Hierarchical MCTS. In Subbarao Kambhampati, editor, *Proceedings of the Twenty-Fifth International Joint Conference on Artificial Intelligence, IJCAI 2016, New York, NY, USA, 9-15 July 2016*, pages 3029–3039, Washington, DC, 2016. IJCAI/AAAI Press.
- [8] Radha-Krishna Balla and Alan Fern. UCT for Tactical Assault Planning in Real-Time Strategy Games. In Craig Boutilier, editor, *IJCAI 2009, Proceedings of the 21st International Joint Conference on Artificial Intelligence, Pasadena, California, USA, July 11-17, 2009*, pages 40–45, 2009.
- [9] Nolan Bard, Michael Johanson, and Michael H. Bowling. Asymmetric abstractions for adversarial settings. In Ana L. C. Bazzan, Michael N. Huhns, Alessio Lomuscio, and Paul Scerri, editors, *International conference on Autonomous Agents and Multi-Agent Systems, AAMAS '14, Paris, France, May 5-9, 2014*, pages 501–508, New York, NY, 2014. IFAAMAS/ACM.
- [10] Nicolas A. Barriga, Marius Stanescu, and Michael Buro. Combining Strategic Learning with Tactical Search in Real-Time Strategy Games. In Brian Magerko and Jonathan P. Rowe, editors, *Proceedings of the Thirteenth AAAI Conference on Artificial Intelligence and Interactive Digital Entertainment (AIIDE-17), October 5-9, 2017, Snowbird, Little Cottonwood Canyon, Utah, USA*, pages 9–15, Washington, DC, 2017. AAAI Press.
- [11] Nicolas A. Barriga, Marius Stanescu, and Michael Buro. Game Tree Search Based on Nondeterministic Action Scripts in Real-Time Strategy Games. *IEEE Trans. Games*, 10(1):69–77, 2018.
- [12] Nicolas Arturo Barriga, Marius Stanescu, and Michael Buro. Puppet Search: Enhancing Scripted Behavior by Look-Ahead Search with Applications to Real-Time Strategy Games. In Arnav Jhala and Nathan R. Sturtevant, editors, *Proceedings of the Eleventh AAAI Conference on Artificial Intelligence and Interactive Digital Entertainment, AIIDE 2015, November 14-18, 2015, University of California, Santa Cruz, CA, USA*, pages 9–15, Washington, DC, 2015. AAAI Press.
- [13] Andrew G. Barto, Steven J. Bradtko, and Satinder P. Singh. Learning to Act Using Real-Time Dynamic Programming. *Artif. Intell.*, 72(1-2):81–138, 1995.
- [14] Sven Behnke. Local Multiresolution Path Planning. In Daniel Polani, Brett Browning, Andrea Bonarini, and Kazuo Yoshida, editors, *RoboCup 2003: Robot Soccer World Cup VII*, volume 3020 of *Lecture Notes in Computer Science*, pages 332–343, Berlin, Heidelberg, 2003. Springer.
- [15] Christopher Berner, Greg Brockman, Brooke Chan, Vicki Cheung, Przemyslaw Debiak, Christy Dennison, David Farhi, Quirin Fischer, Shariq Hashme, Christopher Hesse, Rafal Józefowicz, Scott Gray, Catherine Olsson, Jakub Pachocki, Michael Petrov, Henrique Pondé de Oliveira Pinto, Jonathan Raiman, Tim Salimans, Jeremy Schlatter, Jonas Schneider, Szymon Sidor, Ilya Sutskever, Jie Tang, Filip Wolski, and Susan Zhang. Dota 2 with Large Scale Deep Reinforcement Learning. *CoRR*, abs/1912.06680, 2019.
- [16] Federico Bianchi, Lorenzo Bonanni, Alberto Castellini, and Alessandro Farinelli. Monte Carlo Tree Search Planning for Continuous Action and State Spaces (short paper). In Salvatore Maria Anzalone, Gloria Beraldo, Luca Buoncompagni, Alberto Castellini, and Alberto Finzi, editors, *Proceedings of the 9th Italian Workshop on Artificial Intelligence and Robotics co-located with the 21th International Conference of the Italian Association for Artificial Intelligence, AIRO@AIxIA 2022, Udine, Italy, November 30, 2022*, volume 3417 of *CEUR Workshop Proceedings*, pages 38–47, Aachen, 2022. CEUR-WS.org.
- [17] Darse Billings, Neil Burch, Aaron Davidson, Robert C. Holte, Jonathan Schaeffer, Terence Schauenberg, and Duane Szafron. Approximating Game-Theoretic Optimal Strategies for Full-scale Poker. In Georg Gottlob and Toby Walsh, editors, *IJCAI-03, Proceedings of the Eighteenth International Joint Conference on Artificial Intelligence, Acapulco, Mexico, August 9-15, 2003*, pages 661–668, San Francisco, CA, 2003. Morgan Kaufmann.
- [18] John R. Birge. Chapter 20 Optimization Methods in Dynamic Portfolio Management. In John R. Birge and Vadim Linetsky, editors, *Financial Engineering*, volume 15 of *Handbooks in Operations Research and Management Science*, pages 845–865. Elsevier, New York, NY, 2007.
- [19] Andrea Bolognesi and Paolo Ciancarini. Searching over Metapositions in Kriegspiel. In H. Jaap van den Herik, Yngvi Björnsson, and Nathan S. Netanyahu, editors, *Computers and Games, 4th International Conference, CG 2004, Ramat-Gan, Israel, July 5-7, 2004, Revised Papers*, volume 3846 of *Lecture Notes in Computer Science*, pages 246–261, Berlin, Heidelberg, 2004. Springer.
- [20] Blai Bonet and Bob Givan. Non-Deterministic Planning Track of the 2006 International Planning Competition, 2006. <https://ipc06.icaps-conference.org/probabilistic/> (Accessed: 25.9.2024).
- [21] Craig Boutilier and Richard Dearden. Using Abstractions for Decision-Theoretic Planning with Time Constraints. In Barbara Hayes-Roth and

- Richard E. Korf, editors, *Proceedings of the 12th National Conference on Artificial Intelligence, Seattle, WA, USA, July 31 - August 4, 1994, Volume 2*, pages 1016–1022, Washington, DC, 1994. AAAI Press / The MIT Press.
- [22] Bruno Bouzy. Move-Pruning Techniques for Monte-Carlo Go. In H. Jaap van den Herik, Shun-chin Hsu, Tsan-sheng Hsu, and H. H. L. M. Donkers, editors, *Advances in Computer Games, 11th International Conference, ACG 2005, Taipei, Taiwan, September 6-9, 2005. Revised Papers*, volume 4250 of *Lecture Notes in Computer Science*, pages 104–119, Berlin, Heidelberg, 2006. Springer.
- [23] Bruno Bouzy and Bernard Helmstetter. Monte-Carlo Go Developments. In H. Jaap van den Herik, Hiroyuki Iida, and Ernst A. Heinz, editors, *Advances in Computer Games, Many Games, Many Challenges, 10th International Conference, ACG 2003, Graz, Austria, November 24-27, 2003, Revised Papers*, volume 263 of *IFIP*, pages 159–174, Amsterdam, 2003. Kluwer.
- [24] Alexander Braylan, Mark Hollenbeck, Elliot Meyerson, and Risto Miikkulainen. Frame Skip Is a Powerful Parameter for Learning to Play Atari. In Michael Bowling, Marc G. Bellemare, Erik Talvitie, Joel Veness, and Marlos C. Machado, editors, *Learning for General Competency in Video Games, Papers from the 2015 AAAI Workshop, Austin, Texas, USA, January 26, 2015*, volume WS-15-10 of *AAAI Technical Report*, Washington, DC, 2015. AAAI Press.
- [25] Noam Brown and Tuomas Sandholm. Libratus: The Superhuman AI for No-Limit Poker. In Carles Sierra, editor, *Proceedings of the Twenty-Sixth International Joint Conference on Artificial Intelligence, IJCAI 2017, Melbourne, Australia, August 19-25, 2017*, pages 5226–5228, San Francisco, CA, 2017. ijcai.org.
- [26] Cameron Browne, Edward Jack Powley, Daniel Whitehouse, Simon M. Lucas, Peter I. Cowling, Philipp Rohlfshagen, Stephen Tavenor, Diego Perez Liebana, Spyridon Samothrakis, and Simon Colton. A Survey of Monte Carlo Tree Search Methods. *IEEE Trans. Comput. Intell. AI Games*, 4(1):1–43, 2012.
- [27] Filip Cano, Thomas A. Henzinger, Bettina Könighofer, Konstantin Kueffner, and Kaushik Mallik. Abstraction-Based Decision Making for Statistical Properties (Invited Talk). In Jakob Rehof, editor, *9th International Conference on Formal Structures for Computation and Deduction, FSCD 2024, July 10-13, 2024, Tallinn, Estonia*, volume 299 of *LIPICs*, pages 2:1–2:17, Dagstuhl, Germany, 2024. Schloss Dagstuhl - Leibniz-Zentrum für Informatik.
- [28] Benjamin E. Childs, James H. Brodeur, and Levente Kocsis. Transpositions and move groups in Monte Carlo tree search. In Philip Hingston and Luigi Barone, editors, *Proceedings of the 2008 IEEE Symposium on Computational Intelligence and Games, CIG 2009, Perth, Australia, 15-18 December, 2008*, pages 389–395, New York, NY, 2008. IEEE.
- [29] Michael Chung, Michael Buro, and Jonathan Schaeffer. Monte Carlo Planning in RTS Games. In *Proceedings of the 2005 IEEE Symposium on Computational Intelligence and Games (CIG05), Essex University, Colchester, Essex, UK, 4-6 April, 2005*, New York, NY, 2005. IEEE.
- [30] David Churchill. SparCraft: open source StarCraft combat simulation. <http://code.google.com/p/sparcraft/>. (Accessed: 27.8.2024).
- [31] David Churchill and Michael Buro. Portfolio greedy search and simulation for large-scale combat in starcraft. In *2013 IEEE Conference on Computational Intelligence in Games (CIG), Niagara Falls, ON, Canada, August 11-13, 2013*, pages 1–8, New York, NY, 2013. IEEE.
- [32] David Churchill and Michael Buro. Hierarchical Portfolio Search: Prismata’s Robust AI Architecture for Games with Large Search Spaces. In Arnab Jhala and Nathan R. Sturtevant, editors, *Proceedings of the Eleventh AAAI Conference on Artificial Intelligence and Interactive Digital Entertainment, AIIDE 2015, November 14-18, 2015, University of California, Santa Cruz, CA, USA*, pages 16–22, Washington, DC, 2015. AAAI Press.
- [33] David Churchill, Abdallah Saffidine, and Michael Buro. Fast Heuristic Search for RTS Game Combat Scenarios. In Mark O. Riedl and Gita Sukthankar, editors, *Proceedings of the Eighth AAAI Conference on Artificial Intelligence and Interactive Digital Entertainment, AIIDE-12, Stanford, California, USA, October 8-12, 2012*, Washington, DC, 2012. The AAAI Press.
- [34] Paolo Ciancarini and Gian Piero Favini. Representing Kriegspiel States with Metapositions. In Manuela M. Veloso, editor, *IJCAI 2007, Proceedings of the 20th International Joint Conference on Artificial Intelligence, Hyderabad, India, January 6-12, 2007*, pages 2450–2455, 2007.
- [35] United States Playing Card Company. Blackjack. <https://bicyclecards.com/how-to-play/blackjack> Accessed: 2024-09-28.
- [36] Adrien Couëtoux. *Monte Carlo Tree Search for Continuous and Stochastic Sequential Decision Making Problems*. PhD thesis, University of Paris-Sud, Orsay, France, 2013.
- [37] Adrien Couëtoux, Hassen Doghmen, and Olivier Teytaud. Improving the Exploration in Upper Confidence Trees. In Youssef Hamadi and Marc Schoenauer, editors, *Learning and Intelligent Optimization - 6th International Conference, LION 6, Paris, France, January 16-20, 2012, Revised Selected Papers*, volume 7219 of *Lecture Notes in Computer Science*, pages 366–371, Berlin, Heidelberg, 2012. Springer.
- [38] Adrien Couëtoux, Jean-Baptiste Hoock, Nataliya Sokolovska, Olivier Teytaud, and Nicolas Bonnard. Continuous Upper Confidence Trees. In Carlos A. Coello Coello, editor, *Learning and Intelligent Optimization - 5th International Conference, LION 5, Rome, Italy, January 17-21, 2011. Selected Papers*, volume 6683 of *Lecture Notes in Computer Science*, pages 433–445, Berlin, Heidelberg, 2011. Springer.
- [39] Richard Dearden and Craig Boutilier. Abstraction and Approximate Decision-Theoretic Planning. *Artif. Intell.*, 89(1-2):219–283, 1997.
- [40] Claude G. Diderich and Marc Gengler. A Survey on Minimax Trees and Associated Algorithms. *J. Int. Comput. Games Assoc.*, 17(4):222, 1994.
- [41] Alexander Dockhorn, Christoph Doell, Matthias Hewelt, and Rudolf Kruse. A Decision Heuristic for Monte Carlo Tree Search Doppelkopf Agents. In *2017 IEEE Symposium Series on Computational Intelligence, SSCI 2017, Honolulu, HI, USA, November 27 - Dec. 1, 2017*, pages 1–8, New York, NY, 2017. IEEE.
- [42] Alexander Dockhorn, Max Frick, Ünal Akkaya, and Rudolf Kruse. Predicting Opponent Moves for Improving Hearthstone AI. In *Information Processing and Management of Uncertainty in Knowledge-Based Systems. Theory and Foundations - 17th International Conference, IPMU 2018, Cádiz, Spain, June 11-15, 2018, Proceedings, Part II*, volume 854 of *Communications in Computer and Information Science*, pages 621–632, Berlin, Heidelberg, 2018. Springer.
- [43] Alexander Dockhorn, Jorge Hurtado Grueso, Dominik Jeurissen, and Diego Perez Liebana. STRATEGA: A General Strategy Games Framework. In Joseph C. Osborn, editor, *Joint Proceedings of the AIIDE 2020 Workshops co-located with 16th AAAI Conference on Artificial Intelligence and Interactive Digital Entertainment (AIIDE 2020), Worcester, MA, USA, October 19-23, 2020 (online)*, volume 2862 of *CEUR Workshop Proceedings*, Aachen, Germany, 2020. CEUR-WS.org.
- [44] Alexander Dockhorn, Jorge Hurtado Grueso, Dominik Jeurissen, Linjie Xu, and Diego Perez Liebana. Portfolio Search and Optimization for General Strategy Game-Playing. In *IEEE Congress on Evolutionary Computation, CEC 2021, Kraków, Poland, June 28 - July 1, 2021*, pages 2085–2092, New York, NY, 2021. IEEE.
- [45] Alexander Dockhorn, Jorge Hurtado-Grueso, Dominik Jeurissen, Linjie Xu, and Diego Perez-Liebana. Game State and Action Abstracting Monte Carlo Tree Search for General Strategy Game-Playing. In *2021 IEEE Conference on Games (CoG)*, pages 1–8, 2021.
- [46] Alexander Dockhorn and Rudolf Kruse. *State and Action Abstraction for Search and Reinforcement Learning Algorithms*, pages 181–198. Springer Nature Switzerland, Cham, 2023.
- [47] Robot Entertainment. Hero Academy, 2012.
- [48] Gabriel Van Eyck and Martin Müller. Revisiting Move Groups in Monte-Carlo Tree Search. In H. Jaap van den Herik and Aske Plaat, editors, *Advances in Computer Games - 13th International Conference, ACG 2011, Tilburg, The Netherlands, November 20-22, 2011, Revised Selected Papers*, volume 7168 of *Lecture Notes in Computer Science*, pages 13–23, Berlin, Heidelberg, 2011. Springer.
- [49] Lamia Friha, P. Berry, and Berthe Y Choueiry. DISA: A Distributed scheduler using abstractions. *Revue d’Intelligence Artificielle*, 11, 01 1997.
- [50] Thomas Gabor, Jan Peter, Thomy Phan, Christian Meyer, and Claudia Linnhoff-Popien. Subgoal-Based Temporal Abstraction in Monte-Carlo Tree Search. In Sarit Kraus, editor, *Proceedings of the Twenty-Eighth International Joint Conference on Artificial Intelligence, IJCAI 2019, Macao, China, August 10-16, 2019*, pages 5562–5568, San Francisco, CA, 2019. ijcai.org.
- [51] Martin Gardner. The fantastic combinations of John Conway’s new solitaire game ‘life’. *Scientific American*, 223(4):120–123, October 1970.

- [52] Martin Gardner. Mathematical Games. *Scientific American*, 228(1):108, January 1973.
- [53] Andrew Gilpin and Tuomas Sandholm. Better automated abstraction techniques for imperfect information games, with application to Texas Hold'em poker. In Edmund H. Durfee, Makoto Yokoo, Michael N. Huhns, and Onn Shehory, editors, *6th International Joint Conference on Autonomous Agents and Multiagent Systems (AAMAS 2007)*, Honolulu, Hawaii, USA, May 14-18, 2007, page 192, New York, NY, 2007. IFAAMAS.
- [54] Andrew Gilpin and Tuomas Sandholm. Expectation-Based Versus Potential-Aware Automated Abstraction in Imperfect Information Games: An Experimental Comparison Using Poker. In Dieter Fox and Carla P. Gomes, editors, *Proceedings of the Twenty-Third AAAI Conference on Artificial Intelligence, AAAI 2008, Chicago, Illinois, USA, July 13-17, 2008*, pages 1454–1457, Washington, DC, 2008. AAAI Press.
- [55] Andrew Gilpin, Tuomas Sandholm, and Troels Bjerre Sørensen. A heads-up no-limit Texas Hold'em poker player: discretized betting models and automatically generated equilibrium-finding programs. In Lin Padgham, David C. Parkes, Jörg P. Müller, and Simon Parsons, editors, *7th International Joint Conference on Autonomous Agents and Multiagent Systems (AAMAS 2008)*, Estoril, Portugal, May 12-16, 2008, Volume 2, pages 911–918, New York, NY, 2008. IFAAMAS.
- [56] Matthew L. Ginsberg. Partition Search. In William J. Clancey and Daniel S. Weld, editors, *Proceedings of the Thirteenth National Conference on Artificial Intelligence and Eighth Innovative Applications of Artificial Intelligence Conference, AAAI 96, IAAI 96, Portland, Oregon, USA, August 4-8, 1996, Volume 1*, pages 228–233, Washington, DC, 1996. AAAI Press / The MIT Press.
- [57] Matthew L. Ginsberg. GIB: Steps Toward an Expert-Level Bridge-Playing Program. In Thomas Dean, editor, *Proceedings of the Sixteenth International Joint Conference on Artificial Intelligence, IJCAI 99, Stockholm, Sweden, July 31 - August 6, 1999. 2 Volumes, 1450 pages*, pages 584–593, San Francisco, CA, 1999. Morgan Kaufmann.
- [58] Marek Grzes, Jesse Hoey, and Scott Sanner. International Probabilistic Planning Competition (IPPC) 2014. In *Proceedings of the International Conference on Automated Planning and Scheduling (ICAPS)*, 2014.
- [59] J. T. Guerin, J. P. Hanna, L. Ferland, N. Mattei, and J. Goldsmith. The Academic Advising Planning Domain. In *Proceedings of the Workshop on International Planning Competition (WS-IPC)*, 2012.
- [60] Carlos Guestrin, Daphne Koller, Ronald Parr, and Shobha Venkataraman. Efficient Solution Algorithms for Factored MDPs. *J. Artif. Intell. Res.*, 19:399–468, 2003.
- [61] Eric A. Hansen, Daniel S. Bernstein, and Shlomo Zilberstein. Dynamic Programming for Partially Observable Stochastic Games. In Deborah L. McGuinness and George Ferguson, editors, *Proceedings of the Nineteenth National Conference on Artificial Intelligence, Sixteenth Conference on Innovative Applications of Artificial Intelligence, July 25-29, 2004, San Jose, California, USA*, pages 709–715, Washington, DC, 2004. AAAI Press / The MIT Press.
- [62] Peter E. Hart, Nils J. Nilsson, and Bertram Raphael. A Formal Basis for the Heuristic Determination of Minimum Cost Paths. *IEEE Trans. Syst. Sci. Cybern.*, 4(2):100–107, 1968.
- [63] John Alexander Hawkin, Robert Holte, and Duane Szafron. Automated Action Abstraction of Imperfect Information Extensive-Form Games. In Wolfram Burgard and Dan Roth, editors, *Proceedings of the Twenty-Fifth AAAI Conference on Artificial Intelligence, AAAI 2011, San Francisco, California, USA, August 7-11, 2011*, pages 681–687, Washington, DC, 2011. AAAI Press.
- [64] Ruijie He, Emma Brunskill, and Nicholas Roy. Efficient Planning under Uncertainty with Macro-actions. *J. Artif. Intell. Res.*, 40:523–570, 2011.
- [65] Andreas Hertle, Christian Dornhege, Thomas Keller, Robert Mattmüller, Manuela Ortlieb, and Bernhard Nebel. An Experimental Comparison of Classical, FOND and Probabilistic Planning. In Carsten Lutz and Michael Thielscher, editors, *KI 2014: Advances in Artificial Intelligence - 37th Annual German Conference on AI, Stuttgart, Germany, September 22-26, 2014. Proceedings*, volume 8736 of *Lecture Notes in Computer Science*, pages 297–308, Berlin, Heidelberg, 2014. Springer.
- [66] Jörg Hoffmann and Bernhard Nebel. The FF Planning System: Fast Plan Generation Through Heuristic Search. *J. Artif. Intell. Res.*, 14:253–302, 2001.
- [67] David Hopper and Kenneth Whyld. Kriegspiel. In *The Oxford Companion to Chess*, page 211. Oxford University Press, Oxford, England, 2nd edition, 1996. First published in 1992.
- [68] Jesse Hostetler, Alan Fern, and Thomas G. Dietterich. Progressive Abstraction Refinement for Sparse Sampling. In Marina Meila and Tom Heskes, editors, *Proceedings of the Thirty-First Conference on Uncertainty in Artificial Intelligence, UAI 2015, July 12-16, 2015, Amsterdam, The Netherlands*, pages 365–374, Corvallis, Oregon, 2015. AUAI Press.
- [69] Jesse Hostetler, Alan Fern, and Thomas G. Dietterich. Sample-Based Tree Search with Fixed and Adaptive State Abstractions. *J. Artif. Intell. Res.*, 60:717–777, 2017.
- [70] Jesse Hostetler, Alan Fern, and Tom Dietterich. State Aggregation in Monte Carlo Tree Search. In Carla E. Brodley and Peter Stone, editors, *Proceedings of the Twenty-Eighth AAAI Conference on Artificial Intelligence, July 27 -31, 2014, Québec City, Québec, Canada*, pages 2446–2452, Washington, DC, 2014. AAAI Press.
- [71] Nan Jiang, Satinder Singh, and Richard L. Lewis. Improving UCT planning via approximate homomorphisms. In Ana L. C. Bazzan, Michael N. Huhns, Alessio Lomuscio, and Paul Scerri, editors, *International conference on Autonomous Agents and Multi-Agent Systems, AAMAS '14, Paris, France, May 5-9, 2014*, pages 1289–1296, New York, NY, 2014. IFAAMAS/ACM.
- [72] Michael Johanson. Robust Strategies and Counter-Strategies: Building a Champion Level Computer Poker Player. Master's thesis, University of Alberta, Department of Computing Science, October 2007.
- [73] Michael Johanson, Neil Burch, Richard Anthony Valenzano, and Michael Bowling. Evaluating state-space abstractions in extensive-form games. In Maria L. Gini, Onn Shehory, Takayuki Ito, and Catholijn M. Jonker, editors, *International conference on Autonomous Agents and Multi-Agent Systems, AAMAS '13, Saint Paul, MN, USA, May 6-10, 2013*, pages 271–278, New York, NY, 2013. IFAAMAS.
- [74] Michael Johanson, Kevin Waugh, Michael H. Bowling, and Martin Zinkevich. Accelerating Best Response Calculation in Large Extensive Games. In Toby Walsh, editor, *IJCAI 2011, Proceedings of the 22nd International Joint Conference on Artificial Intelligence, Barcelona, Catalonia, Spain, July 16-22, 2011*, pages 258–265, Washington, DC, 2011. IJCAI/AAAI.
- [75] Niels Justesen, Balint Tillman, Julian Togelius, and Sebastian Risi. Script- and cluster-based UCT for StarCraft. In *2014 IEEE Conference on Computational Intelligence and Games, CIG 2014, Dortmund, Germany, August 26-29, 2014*, pages 1–8, New York, NY, 2014. IEEE.
- [76] Shivaram Kalyanakrishnan, Siddharth Aravindan, Vishwajeet Bagawat, Varun Bhatt, Harshith Goka, Archit Gupta, Kalpesh Krishna, and Vihari Piratla. An Analysis of Frame-skipping in Reinforcement Learning. *CoRR*, abs/2102.03718, 2021.
- [77] Richard E. Korf. Depth-First Iterative-Deepening: An Optimal Admissible Tree Search. *Artif. Intell.*, 27(1):97–109, 1985.
- [78] Levi H. S. Lelis. Stratified Strategy Selection for Unit Control in Real-Time Strategy Games. In Carles Sierra, editor, *Proceedings of the Twenty-Sixth International Joint Conference on Artificial Intelligence, IJCAI 2017, Melbourne, Australia, August 19-25, 2017*, pages 3735–3741, San Francisco, CA, 2017. ijcai.org.
- [79] Diego Perez Liebana, Alexander Dockhorn, Jorge Hurtado Grueso, and Dominik Jurisissen. The Design Of "Stratega": A General Strategy Games Framework. *CoRR*, abs/2009.05643, 2020.
- [80] Diego Perez Liebana, Spyridon Samothrakis, Simon M. Lucas, and Philipp Rohlfshagen. Rolling horizon evolution versus tree search for navigation in single-player real-time games. In Christian Blum and Enrique Alba, editors, *Genetic and Evolutionary Computation Conference, GECCO '13, Amsterdam, The Netherlands, July 6-10, 2013*, pages 351–358, New York, NY, 2013. ACM.
- [81] Iain Little and Sylvie Thiebaux. Probabilistic Planning vs Replanning. In *Proceedings of the ICAPS Workshop on Planning Competitions: Past, Present, and Future*, 2007.
- [82] Qi Liu, Xueyuan Li, Shihua Yuan, and Zirui Li. Decision-Making Technology for Autonomous Vehicles: Learning-Based Methods, Applications and Future Outlook. In *24th IEEE International Intelligent Transportation Systems Conference, ITSC 2021, Indianapolis, IN, USA, September 19-22, 2021*, pages 30–37, New York, NY, 2021. IEEE.
- [83] Derek Long, Stephen F. Smith, Daniel Borrajo, and Lee McCluskey, editors. *Proceedings of the Sixteenth International Conference on Automated Planning and Scheduling, ICAPS 2006, Cumbria, UK, June 6-10, 2006*, Washington, DC, 2006. AAAI.
- [84] Richard J. Lorentz. Amazons Discover Monte-Carlo. In *Proceedings of the 6th International Conference on Computers and Games (CG'08)*, pages 13–24, Berlin, Heidelberg, 2008. Springer-Verlag.

- [85] Julian R. H. Mariño, Rubens O. Moraes, Cláudio Toledo, and Levi H. S. Lelis. Evolving Action Abstractions for Real-Time Planning in Extensive-Form Games. In *The Thirty-Third AAAI Conference on Artificial Intelligence, AAAI 2019, The Thirty-First Innovative Applications of Artificial Intelligence Conference, IAAI 2019, The Ninth AAAI Symposium on Educational Advances in Artificial Intelligence, EAAI 2019, Honolulu, Hawaii, USA, January 27 - February 1, 2019*, pages 2330–2337, Washington, DC, 2019. AAAI Press.
- [86] Kenrick J. Mock. Hierarchical Heuristic Search Techniques for Empire-based Games. In Hamid R. Arabnia and Youngsong Mun, editors, *Proceedings of the International Conference on Artificial Intelligence, IC-AI '02, June 24 - 27, 2002, Las Vegas, Nevada, USA, Volume 2*, pages 643–648, Las Vegas, Nevada, 2002. CSREA Press.
- [87] Rubens Moraes, Julian Mariño, and Levi Lelis. Nested-Greedy Search for Adversarial Real-Time Games. *Proceedings of the AAAI Conference on Artificial Intelligence and Interactive Digital Entertainment*, 14(1):67–73, Sep. 2018.
- [88] Rubens O. Moraes and Levi H. S. Lelis. Asymmetric Action Abstractions for Multi-Unit Control in Adversarial Real-Time Games. In Sheila A. McIlraith and Kilian Q. Weinberger, editors, *Proceedings of the Thirty-Second AAAI Conference on Artificial Intelligence (AAAI-18), the 30th innovative Applications of Artificial Intelligence (IAAI-18), and the 8th AAAI Symposium on Educational Advances in Artificial Intelligence (EAAI-18), New Orleans, Louisiana, USA, February 2-7, 2018*, pages 876–883, Washington, DC, 2018. AAAI Press.
- [89] Rubens O. Moraes, Julian R. H. Mariño, Levi H. S. Lelis, and Mario A. Nascimento. Action Abstractions for Combinatorial Multi-Armed Bandit Tree Search. In Jonathan P. Rowe and Gillian Smith, editors, *Proceedings of the Fourteenth AAAI Conference on Artificial Intelligence and Interactive Digital Entertainment, AIIDE 2018, November 13-17, 2018, Edmonton, Canada*, pages 74–80, Washington, DC, 2018. AAAI Press.
- [90] Rubens O. Moraes, Mario A. Nascimento, and Levi H. S. Lelis. Asymmetric Action Abstractions for Planning in Real-Time Strategy Games. *J. Artif. Intell. Res.*, 75:1103–1137, 2022.
- [91] Samer B. Nashed, Justin Svegliato, Matteo Brucato, Connor Basich, Rod Grupen, and Shlomo Zilberstein. Solving Markov Decision Processes with Partial State Abstractions. In *IEEE International Conference on Robotics and Automation, ICRA 2021, Xi'an, China, May 30 - June 5, 2021*, pages 813–819, New York, NY, 2021. IEEE.
- [92] Santiago Ontañón and Michael Buro. Adversarial Hierarchical-Task Network Planning for Complex Real-Time Games. In Qiang Yang and Michael J. Wooldridge, editors, *Proceedings of the Twenty-Fourth International Joint Conference on Artificial Intelligence, IJCAI 2015, Buenos Aires, Argentina, July 25-31, 2015*, pages 1652–1658, Washington, DC, 2015. AAAI Press.
- [93] Santiago Ontañón. The Combinatorial Multi-Armed Bandit Problem and its Application to Real-Time Strategy Games. In *Proceedings of the Ninth AAAI Conference on Artificial Intelligence and Interactive Digital Entertainment (AIIDE 2013)*, pages 58–64, 2013.
- [94] Christos H. Papadimitriou and Mihalis Yannakakis. Shortest Paths Without a Map. *Theor. Comput. Sci.*, 84(1):127–150, 1991.
- [95] Jason Pazis and Michail G. Lagoudakis. Binary action search for learning continuous-action control policies. In Andrea Pohoreckýj Danyluk, Léon Bottou, and Michael L. Littman, editors, *Proceedings of the 26th Annual International Conference on Machine Learning, ICML 2009, Montreal, Quebec, Canada, June 14-18, 2009*, volume 382 of *ACM International Conference Proceeding Series*, pages 793–800, New York, NY, 2009. ACM.
- [96] Jason Pazis and Michail G. Lagoudakis. Learning continuous-action control policies. In *IEEE Symposium on Adaptive Dynamic Programming and Reinforcement Learning, ADPRL 2009, Nashville, TN, USA, March 31 - April 1, 2009*, pages 169–176, New York, NY, 2009. IEEE.
- [97] Diego Pérez-Liébana, Cristina Guerrero-Romero, Alexander Dockhorn, Linjie Xu, Jorge Hurtado Grueso, and Dominik Jeurissen. Generating Diverse and Competitive Play-Styles for Strategy Games. In *2021 IEEE Conference on Games (CoG), Copenhagen, Denmark, August 17-20, 2021*, pages 1–8, New York, NY, 2021. IEEE.
- [98] Diego Pérez-Liébana, Yu-Jhen Hsu, Stavros Emmanouilidis, Bobby Dewan Akram Khaleque, and Raluca D. Gaina. Tribes: A New Turn-Based Strategy Game for AI Research. In Levi Lelis and David Thue, editors, *Proceedings of the Sixteenth AAAI Conference on Artificial Intelligence and Interactive Digital Entertainment, AIIDE 2020, virtual, October 19-23, 2020*, pages 252–258, Washington, DC, 2020. AAAI Press.
- [99] B. Ravindran and A. G. Barto. Approximate Homomorphisms: A Framework for Non-Exact Minimization in Markov Decision Processes. In *Proc. Int. Conf. Knowl.-Based Comput. Syst.*, pages 1–10, 2004.
- [100] Balaraman Ravindran and Andrew G. Barto. SMDP Homomorphisms: An Algebraic Approach to Abstraction in Semi-Markov Decision Processes. In Georg Gottlob and Toby Walsh, editors, *IJCAI-03, Proceedings of the Eighteenth International Joint Conference on Artificial Intelligence, Acapulco, Mexico, August 9-15, 2003*, pages 1011–1018, San Francisco, CA, 2003. Morgan Kaufmann.
- [101] Franisek Sailer, Michael Buro, and Marc Lanctot. Adversarial Planning Through Strategy Simulation. In *Proceedings of the 2007 IEEE Symposium on Computational Intelligence and Games, CIG 2007, Honolulu, Hawaii, USA, 1-5 April, 2007*, pages 80–87, New York, NY, 2007. IEEE.
- [102] S. Saisubramanian, S. Zilberstein, and P. Shenoy. Optimizing Electric Vehicle Charging Through Determinization. In *ICAPS Workshop on Scheduling and Planning Applications*, 2017.
- [103] Sandhya Saisubramanian and Shlomo Zilberstein. Adaptive Outcome Selection for Planning with Reduced Models. In *2019 IEEE/RSJ International Conference on Intelligent Robots and Systems, IROS 2019, Macau, SAR, China, November 3-8, 2019*, pages 1655–1660, New York, NY, 2019. IEEE.
- [104] J.-T. Saito, M. H. Winands, J. W. Uiterwijk, and H. J. van den Herik. Grouping Nodes for Monte-Carlo Tree Search. In *Computer Games Workshop*, pages 276–283, 2007.
- [105] Makoto Sakuta and Hiroyuki Iida. Solving Kriegspiel-like Problems: Exploiting a Transposition Table. *J. Int. Comput. Games Assoc.*, 23(4):218–229, 2000.
- [106] Tuomas Sandholm and Satinder Singh. Lossy stochastic game abstraction with bounds. In Boi Faltings, Kevin Leyton-Brown, and Panos Ipeirotis, editors, *Proceedings of the 13th ACM Conference on Electronic Commerce, EC 2012, Valencia, Spain, June 4-8, 2012*, pages 880–897, New York, NY, 2012. ACM.
- [107] Scott Sanner and Sungwook Yoon. International Probabilistic Planning Competition (IPPC) 2011. In *Proceedings of the International Conference on Automated Planning and Scheduling (ICAPS)*, 2011.
- [108] David Schnizlein. State translation in no-limit poker. Master's thesis, University of Alberta, Department of Computing Science, Fall 2009.
- [109] David Schnizlein, Michael H. Bowling, and Duane Szafron. Probabilistic State Translation in Extensive Games with Large Action Sets. In Craig Boutilier, editor, *IJCAI 2009, Proceedings of the 21st International Joint Conference on Artificial Intelligence, Pasadena, California, USA, July 11-17, 2009*, pages 278–284, 2009.
- [110] Jiefu Shi and Michael L. Littman. Abstraction Methods for Game Theoretic Poker. In T. Anthony Marsland and Ian Frank, editors, *Computers and Games, Second International Conference, CG 2000, Hamamatsu, Japan, October 26-28, 2000, Revised Papers*, volume 2063 of *Lecture Notes in Computer Science*, pages 333–345, Berlin, Heidelberg, 2000. Springer.
- [111] Cleyton R. Silva, Rubens O. Moraes, Levi H. S. Lelis, and Kobi Gal. Strategy Generation for Multiunit Real-Time Games via Voting. *IEEE Trans. Games*, 11(4):426–435, 2019.
- [112] David Silver, Aja Huang, Chris J. Maddison, Arthur Guez, Laurent Sifre, George van den Driessche, Julian Schrittwieser, Ioannis Antonoglou, Vedavyas Panneshelvam, Marc Lanctot, Sander Dieleman, Dominik Grewe, John Nham, Nal Kalchbrenner, Ilya Sutskever, Timothy P. Lillicrap, Madeleine Leach, Koray Kavukcuoglu, Thore Graepel, and Demis Hassabis. Mastering the game of Go with deep neural networks and tree search. *Nat.*, 529(7587):484–489, 2016.
- [113] Trey Smith and Reid G. Simmons. Heuristic Search Value Iteration for POMDPs. In David Maxwell Chickering and Joseph Y. Halpern, editors, *UAI '04, Proceedings of the 20th Conference in Uncertainty in Artificial Intelligence, Banff, Canada, July 7-11, 2004*, pages 520–527, Corvallis, Oregon, 2004. AUAI Press.
- [114] Tomah Sogabe, Dinesh Bahadur Malla, Shota Takayama, Seiichi Shin, Katsuyoshi Sakamoto, Koichi Yamaguchi, Thakur Praveen Singh, Masaru Sogabe, Tomohiro Hirata, and Yoshitaka Okada. Smart Grid Optimization by Deep Reinforcement Learning over Discrete and Continuous Action Space. In *2018 IEEE 7th World Conference on Photovoltaic Energy Conversion (WCPEC) (A Joint Conference of 45th IEEE PVSC, 28th PVSEC and 34th EU PVSEC)*, pages 3794–3796, 2018.

- [115] Samuel Sokota, Caleb Ho, Zaheen Farraz Ahmad, and J. Zico Kolter. Monte Carlo Tree Search With Iteratively Refining State Abstractions. In Marc'Aurelio Ranzato, Alina Beygelzimer, Yann N. Dauphin, Percy Liang, and Jennifer Wortman Vaughan, editors, *Advances in Neural Information Processing Systems 34: Annual Conference on Neural Information Processing Systems 2021, NeurIPS 2021, December 6-14, 2021, virtual*, pages 18698–18709, 2021.
- [116] Finnegan Southey, Michael Bowling, Bryce Larson, Carmelo Piccione, Neil Burch, Darse Billings, and D. Chris Rayner. Bayes' Bluff: Opponent Modelling in Poker. *CoRR*, abs/1207.1411, 2012.
- [117] Marius Stanescu, Nicolas A. Barriga, and Michael Buro. Hierarchical Adversarial Search Applied to Real-Time Strategy Games. In Ian Horswill and Arnav Jhala, editors, *Proceedings of the Tenth AAAI Conference on Artificial Intelligence and Interactive Digital Entertainment, AIIDE 2014, October 3-7, 2014, North Carolina State University, Raleigh, NC, USA*, Washington, DC, 2014. AAAI.
- [118] Marius Stanescu, Nicolas A. Barriga, Andy Hess, and Michael Buro. Evaluating real-time strategy game states using convolutional neural networks. In *IEEE Conference on Computational Intelligence and Games, CIG 2016, Santorini, Greece, September 20-23, 2016*, pages 1–7, New York, NY, 2016. IEEE.
- [119] Rolf A. N. Starre, Marco Loog, and Frans A. Oliehoek. Model-Based Reinforcement Learning with State Abstraction: A Survey. In Toon Calders, Celine Vens, Jeffrey Lijffijt, and Bart Goethals, editors, *Artificial Intelligence and Machine Learning - 34th Joint Benelux Conference, BNAIC/Benelearn 2022, Mechelen, Belgium, November 7-9, 2022, Revised Selected Papers*, volume 1805 of *Communications in Computer and Information Science*, pages 133–148, Berlin, Heidelberg, 2022. Springer.
- [120] Anthony Stentz. Optimal and Efficient Path Planning for Unknown and Dynamic Environments. Technical Report CMU-RI-TR-93-20, Carnegie Mellon Robotics Institute, August 1993.
- [121] Anthony Stentz. Optimal and Efficient Path Planning for Partially-Known Environments. In *Proceedings of the 1994 International Conference on Robotics and Automation, San Diego, CA, USA, May 1994*, pages 3310–3317, Washington, DC, 1994. IEEE Computer Society.
- [122] Peter Stone, Richard S. Sutton, and Gregory Kuhlmann. Reinforcement Learning for RoboCup Soccer Keepaway. *Adaptive Behavior*, 13(3):165–188, 2005.
- [123] Lunarch Studios. Prismata, 2015.
- [124] Richard S Sutton. Generalization in reinforcement learning: Successful examples using sparse coarse coding. In D. Touretzky, M.C. Mozer, and M. Hasselmo, editors, *Advances in Neural Information Processing Systems*, volume 8, pages 1038–1044, Denver, CO, 1995. MIT Press.
- [125] Richard S. Sutton and Andrew G. Barto. *Reinforcement learning - an introduction*. Adaptive computation and machine learning. MIT Press, Cambridge, MA, 1998.
- [126] Marco Tamassia, Fabio Zambetta, William L. Raffe, Florian 'Floyd' Mueller, and Xiaodong Li. Dynamic Choice of State Abstraction in Q-Learning. In Gal A. Kaminka, Maria Fox, Paolo Bouquet, Eyke Hüllermeier, Virginia Dignum, Frank Dignum, and Frank van Harmelen, editors, *ECAI 2016 - 22nd European Conference on Artificial Intelligence, 29 August-2 September 2016, The Hague, The Netherlands - Including Prestigious Applications of Artificial Intelligence (PAIS 2016)*, volume 285 of *Frontiers in Artificial Intelligence and Applications*, pages 46–54, Amsterdam, Netherlands, 2016. IOS Press.
- [127] Yunhao Tang and Shipra Agrawal. Discretizing Continuous Action Space for On-Policy Optimization. In *The Thirty-Fourth AAAI Conference on Artificial Intelligence, AAAI 2020, The Thirty-Second Innovative Applications of Artificial Intelligence Conference, IAAI 2020, The Tenth AAAI Symposium on Educational Advances in Artificial Intelligence, EAAI 2020, New York, NY, USA, February 7-12, 2020*, pages 5981–5988, Washington, DC, 2020. AAAI Press.
- [128] Jonathan Tash and Stuart Russell. Control Strategies for a Stochastic Planner. In Barbara Hayes-Roth and Richard E. Korf, editors, *Proceedings of the 12th National Conference on Artificial Intelligence, Seattle, WA, USA, July 31 - August 4, 1994, Volume 2*, pages 1079–1085, Washington, DC, 1994. AAAI Press / The MIT Press.
- [129] The Wargus Team. Wargus, 2005. <http://wargus.sourceforge.net/> Accessed: 2024-09-28.
- [130] Alan M. Thompson. The Navigation System of the JPL Robot. In Raj Reddy, editor, *Proceedings of the 5th International Joint Conference on Artificial Intelligence. Cambridge, MA, USA, August 22-25, 1977*, pages 749–757, San Francisco, California, 1977. William Kaufmann.
- [131] Alberto Uriarte and Santiago Ontañón. Game-Tree Search over High-Level Game States in RTS Games. In Ian Horswill and Arnav Jhala, editors, *Proceedings of the Tenth AAAI Conference on Artificial Intelligence and Interactive Digital Entertainment, AIIDE 2014, October 3-7, 2014, North Carolina State University, Raleigh, NC, USA*, Washington, DC, 2014. AAAI.
- [132] R. Vanderbei. Optimal Sailing Strategies. Technical report, University of Princeton, Statistics and Operations Research Program, 1996.
- [133] Thomas J. Walsh, Sergiu Goschin, and Michael L. Littman. Integrating Sample-Based Planning and Model-Based Reinforcement Learning. In Maria Fox and David Poole, editors, *Proceedings of the Twenty-Fourth AAAI Conference on Artificial Intelligence, AAAI 2010, Atlanta, Georgia, USA, July 11-15, 2010*, pages 612–617, Washington, DC, 2010. AAAI Press.
- [134] Che Wang, Pan Chen, Yuanda Li, Christoffer Holmgård, and Julian Togelius. Portfolio Online Evolution in StarCraft. In Nathan R. Sturtevant and Brian Magerko, editors, *Proceedings of the Twelfth AAAI Conference on Artificial Intelligence and Interactive Digital Entertainment, AIIDE 2016, October 8-12, 2016, Burlingame, California, USA*, pages 114–121, Washington, DC, 2016. AAAI Press.
- [135] Yizao Wang, Jean-Yves Audibert, and Rémi Munos. Algorithms for Infinitely Many-Armed Bandits. In Daphne Koller, Dale Schuurmans, Yoshua Bengio, and Léon Bottou, editors, *Advances in Neural Information Processing Systems 21, Proceedings of the Twenty-Second Annual Conference on Neural Information Processing Systems, Vancouver, British Columbia, Canada, December 8-11, 2008*, pages 1729–1736, San Francisco, CA, 2008. Curran Associates, Inc.
- [136] Kevin Waugh, David Schnizlein, Michael H. Bowling, and Duane Szafron. Abstraction pathologies in extensive games. In Carles Sierra, Cristiano Castelfranchi, Keith S. Decker, and Jaime Simão Sichman, editors, *8th International Joint Conference on Autonomous Agents and Multiagent Systems (AAMAS 2009), Budapest, Hungary, May 10-15, 2009, Volume 2*, pages 781–788, New York, NY, 2009. IFAAMAS.
- [137] Kevin Waugh, Martin Zinkevich, Michael Johanson, Morgan Kan, David Schnizlein, and Michael H. Bowling. A Practical Use of Imperfect Recall. In Vadim Bulitko and J. Christopher Beck, editors, *Eighth Symposium on Abstraction, Reformulation, and Approximation, SARA 2009, Lake Arrowhead, California, USA, 8-10 August 2009*, Washington, DC, 2009. AAAI.
- [138] Linjie Xu, Alexander Dockhorn, and Diego Perez-Liebana. Towards Applicable State Abstractions: a Preview in Strategy Games. In *Reinforcement Learning and Decision Making - RL as a Model of Agency Workshop*, 2022.
- [139] Linjie Xu, Alexander Dockhorn, and Diego Perez-Liebana. Elastic Monte Carlo Tree Search. *IEEE Transactions on Games*, 15(4):527–537, 2023.
- [140] Linjie Xu, Diego Perez-Liebana, and Alexander Dockhorn. Strategy Game-Playing with Size-Constrained State Abstraction. In *Proceedings of the IEEE Conference on Games 2024*, pages 1–8, 2024.
- [141] Zuoqi Yang and Santiago Ontañón. Guiding Monte Carlo Tree Search by Scripts in Real-Time Strategy Games. In Gillian Smith and Levi Lelis, editors, *Proceedings of the Fifteenth AAAI Conference on Artificial Intelligence and Interactive Digital Entertainment, AIIDE 2019, October 8-12, 2019, Atlanta, Georgia, USA*, pages 100–107, Washington, DC, 2019. AAAI Press.
- [142] Sung Wook Yoon, Alan Fern, and Robert Givan. FF-Replan: A Baseline for Probabilistic Planning. In Mark S. Boddy, Maria Fox, and Sylvie Thiébaux, editors, *Proceedings of the Seventeenth International Conference on Automated Planning and Scheduling, ICAPS 2007, Providence, Rhode Island, USA, September 22-26, 2007*, page 352, Washington, DC, 2007. AAAI.
- [143] Sung Wook Yoon, Alan Fern, Robert Givan, and Subbarao Kambhampati. Probabilistic Planning via Determinization in Hindsight. In Dieter Fox and Carla P. Gomes, editors, *Proceedings of the Twenty-Third AAAI Conference on Artificial Intelligence, AAAI 2008, Chicago, Illinois, USA, July 13-17, 2008*, pages 1010–1016, Washington, DC, 2008. AAAI Press.
- [144] Håkan L. S. Younes, Michael L. Littman, David Weissman, and John Asmuth. The First Probabilistic Track of the International Planning Competition. *J. Artif. Intell. Res.*, 24:851–887, 2005.
- [145] Walter Zamkaskas. El Juego de las Amazonas. Issue number 4 of the puzzle magazine El Acertijo, December 1992.

- [146] Yu Zhang, Peter Tiño, Ales Leonardis, and Ke Tang. A Survey on Neural Network Interpretability. *IEEE Trans. Emerg. Top. Comput. Intell.*, 5(5):726–742, 2021.
- [147] Martin Zinkevich, Michael H. Bowling, and Neil Burch. A New Algorithm for Generating Equilibria in Massive Zero-Sum Games. In *Proceedings of the Twenty-Second AAAI Conference on Artificial Intelligence, July 22-26, 2007, Vancouver, British Columbia, Canada*, pages 788–794, Washington, DC, 2007. AAAI Press.
- [148] Martin Zinkevich, Michael Johanson, Michael H. Bowling, and Carmelo Piccione. Regret Minimization in Games with Incomplete Information. In John C. Platt, Daphne Koller, Yoram Singer, and Sam T. Roweis, editors, *Advances in Neural Information Processing Systems 20, Proceedings of the Twenty-First Annual Conference on Neural Information Processing Systems, Vancouver, British Columbia, Canada, December 3-6, 2007*, pages 1729–1736, San Francisco, CA, 2007. Curran Associates, Inc.

VII. APPENDIX

A. Benchmark problems

In this subsection, we will be giving a short description of test/benchmark problems that were used by the papers presented in this survey. Usually, for each problem, there are several variants that have been used for experiments. For a more detailed description of each environment and the exact variant used, one has to access the respective paper.

1) *Perfect information, discrete action, single-agent problems*: Most of the problems presented in this category stem from the International Probabilistic Planning Competition (IPPC) [58] which is a competition that is held at the International Conference on Automated Planning and Scheduling (ICAPS) [83]. The problems shown here are only those IPPC problems that were used as testbeds for algorithms discussed in this survey, however, the IPPC features many more such problems.

- **Gridworld**: Gridworld describes a family of problems where the agent's current state is described as a position on a 2-dimensional grid. Each grid cell either contains a wall or it doesn't. At each step, the agent may move to any of the adjacent unoccupied cells, however, this may randomly fail and the agent moves in a random direction. Usually, the agent receives a negative reward at each step. The goal depends on the concrete problem formulation but might be to just reach one goal cell [7] or a set of goal cells [100]. Some researchers also use a continuous version of Gridworld, where the state space is a subset of $\mathbb{R}^2$, and each of the four available actions modifies the state by one plus some Gaussian noise in the chosen direction [7].
- **Sailing Wind**: Originally proposed by Robert Vanderbei [132], the goal of Sailing Wind is to move a ship that starts at $(1, 1)$ on an $n \times n$ grid to (n, n) with minimal cost. There is no consistent use of a transition and reward function throughout the literature. There may just be two available actions (*down*, *right*) [71] or up to seven (each adjacent cell except the one facing a stochastic wind direction) [4]. The cost of each action is dependent on the current wind direction which stochastically changes its direction at each step independent of the player's actions.
- **Game of Life**: The original game of life by John Conway [51] is a cellular automaton and modified into a stochastic MDP as a test problem for the International Probabilistic Planning Competition [107] by introducing noise to the deterministic state transition, setting the current number of alive cells as the reward, and allowing the agent to choose one cell per round that will survive to the next round. States are elements in $\{0, 1\}^{n \times n}$ describing whether there is an alive cell at each cell on a grid.
- **SysAdmin**: Proposed by Guestrin et al. [60], a SysAdmin instance is a graph (describing a network topology) with $n \in \mathbb{N}$ vertices. The state space is $\{0, 1\}^n$ (describing which machines are currently operating) and the action space is $\{1, \dots, n\}$ (describing with machine to reboot). At each step, the reward is dependent on the machines that are currently working, a reboot causes the rebooted machine to have a high chance of working in the next step. Machines can randomly fail at each step, however this probability is increased when a neighbor fails.
- **Navigation**: Navigation was a test problem for the International Probabilistic Planning Competition 2011 [107]. The goal is to move a robot on an $n \times m$ grid from $(n, 1)$ to (n, m) in the least number of steps. The robot may move to any of the four adjacent tiles, however, each tile is assigned a unique probability with which the robot is reset back to $(n, 1)$.
- **Academic Advising**: The Academic Advising domain was introduced by Guerin et al. [59] and used for the IPPC 2014 [58]. The agent is a student whose goal is to pass certain academic classes. Formally, the state is an element in $\{H, L, F, NT\}^n$ (representing for each course whether it has not been taken, failed, or passed with a low or high grade) and the agent's action is to choose a course to take. The course outcome depends on the states of the prerequisite courses. The reward at each step depends on whether and with what success a set of goal states has been taken.
- **Crossing Traffic**: This is another IPPC 2014 [58] grid-based navigation problem where the agent has to maneuver through lanes of traffic. Obstacles can move only on the x -axis from right to left. Obstacles spawn randomly on the right end of the grid and on collision with the agent, render the agent unable to move, thus making the current episode impossible to be solved. The goal is to cross all lanes in the shortest path possible.
- **Tamarisk**: Tamarisk is yet another problem from the IPPC 2014 [58] which models the expansion of an invasive plant in a river system. The river system is modelled as a chain of reaches where each reach contains a number of slots that may be unoccupied, occupied by a native plant, or occupied by the invasive Tamarisk plant. Both plant types spread stochastically to neighboring states with a higher probability of spreading downstream. At each time step, the agent chooses an action for each reach, which are doing nothing, eradicating Tamarisk, or restoring a native plant. The action chosen at a reach is applied to all slots in that reach. Except for the do-nothing action, all actions can randomly fail.
- **Racetrack**: Racetrack was first described by Martin Gardner [52] and later formalized by Barto et al. [13] where the goal is to move a car in as few steps as possible to a goal-position on a graph starting from a random position. Each state is a tuple in $\mathbb{Z}^2 \times \mathbb{Z}^2$ that represents the current position and velocity vector. The available actions are adding a vector from $\{-1, 0, 1\}^2$ to the velocity vector (i.e. accelerating). At each step, with some probability, the chosen acceleration vector is replaced by $(0, 0)$. The state transitions by first adding the acceleration vector to the current velocity vector and then adding the velocity vector to the position vector. If a boundary or an obstacle has been hit, the car's position is reset to

one of the initial positions with zero velocity. Some implementations however, set the velocity vector to zero and place the car at an empty tile closest to the crash position that is on the straight line connecting the last position to the crash position [5].

- **EarthObservation:** This problem was proposed by Hertle et al. [65] which models a satellite orbiting earth. Formally, each state is a position on a 2-dimensional grid, representing the satellite's longitudinal position and the latitude the camera is aimed at as well as weather levels for some designated cells. At each step, the weather levels stochastically change independent of the agent's actions which are to idle, to take a photo of the current position, or increment/decrement the current cells y -position (i.e. shifting the camera focus). A reward is obtained if one of the designated cells is photographed with an amount depending on the cell's current weather condition.
- **Tireworld:** Tireworld was proposed as a test problem for the IPPC 2004 [144]. In the original goal-based version, the agent is a car that traverses a graph. At each step, the car may move to an adjacent node, change its tire, or load a tire. The goal is to reach a designated goal node. At each step, the car's tire may randomly break. If the car isn't carrying a spare tire, the goal can no longer be reached. Otherwise, if available, a spare tire (at most one can be carried) must be used to replace the current tire. Some nodes contain spare tires, which when the agent visits them, can be picked up. In an infinite horizon version, there is no goal position but the agent is dealt a big penalty whenever the tire breaks while the agent does not have a spare one.
- **Auctioneer:** Cano et al. [27] describe a set of problems that we call Auctioneer. The agent controls an auctioneer (or server) that at each step receives n random integer bids from n clients. The available actions are choosing which client's bid to accept. The immediate reward may differ on the concrete formulation and might just be equal to the accepted bid or additionally punish the server if one client is consistently picked more often than other clients.
- **Coffee:** Coffee is a toy problem, proposed by Dearden and Boutilier [39] which models a robot bringing coffee to a user. Each state is described by nine binary variables and one with four values i.e. $S = \{0, 1\}^9 \times \{0, 1, 2, 3\}$. Among other things, the variables describe the robot's current location, whether it is raining, or whether the user has a coffee. At each state, the robot has seven actions including the move action, deliver coffee, or the buy coffee action which have different effects depending on the current state. To solve the task, the robot has to move between an office and a coffee shop, buy coffee, and deliver the coffee to the user whilst staying wet which can be accomplished even if it is raining by taking an umbrella from the office.
- **Builder:** This is another toy problem by Dearden and Boutilier [39] in which the agent is tasked to join together two objects. Each state is given by nine binary variables i.e. $S = \{0, 1\}^9$ which describe whether each object is cleaned, whether they are painted, whether they are shaped, whether they have been drilled, and whether they have been joined. There are ten available actions per state, which include painting or cleaning an object. In particular, there are two ways of joining the objects, either by gluing them together which has a high chance of failure and leaving the objects unclean, or by bolting them together with a high rate of success but this requires the objects to be drilled first. The maximum reward can be obtained when the two objects are joined, cleaned, and painted.
- **Blocksworld:** Blocksworld first appeared in the IPPC 2004 [144] and models stacks of blocks being placed on top of a table. The goal is to stack the blocks into a goal configuration using two types of actions which are picking up and placing down a block. Stochasticity is featured in that there is a chance of blocks slipping, causing the place-down action to place the block on the table. Variations of Blocksworld include colored blocks where goal configuration only specifies color configurations hence there are multiple correct goal configurations or an exploding Blocksworld version where each block can (at most once) detonate during the place-down action and destroy the object it was placed on. This version can result in a dead-end when for example the table or a block of the goal configuration is destroyed.
- **Boxworld:** This is another problem from the IPPC 2004 [144] which models a logistics problem of delivering boxes to their destination. A Boxworld instance is a graph whose nodes represent cities and whose edges either denote that a truck or a plane can move between these cities. The boxes can be delivered through only six action types *load-box-on-truck-in-city*, *unload-box-from-truck-in-city*, *load-box-on-plane-in-city*, *unload-box-from-plane-in-city*, *drive-truck*, and *fly-plane*. Using the plane incurs higher costs than using the truck. However, the truck's movement is not deterministic.
- **Fileworld:** This is a problem from the IPPC 2004 [144] with the goal of filing n files into m folders. The available actions are determining the type of a file (a necessary condition before it can be filed), retrieving and returning any folder, and placing a file into a retrieved folder. The first time the type of a file is retrieved it is assigned the type of any of the m folders with uniform probability. Each action incurs a cost with the highest cost being incurred by the folder-retrieving action.
- **Towers of Hanoi:** This is yet another problem from the IPPC 2004 [144] which is a noisy version of Towers of Hanoi. Like in the original, the goal is to move all five disks on the first peg to the last of three pegs. In this version, one has the option to move two disks at once. Both moving a single disk and two disks has a probability of destroying a disk that is dependent on whether the biggest disk has been moved yet. In case a disk is destroyed, the episode terminates immediately.
- **Zeno travel:** This problem was also featured in the IPPC 2004 [144]. The task is to move travelers to their respective

destinations using planes. Planes require fuel to fly and burn more fuel the faster they fly. The agent’s available action types are whether to board people, whether to debark, when to fly, and whether to speed up during flight. To model durative actions, each action has a corresponding *complete-action* primitive that has a chance of failure. Hence, to realize the effect of any action, the *complete-action* primitive has to possibly be called several times.

- **Drive:** This is a problem from the IPPC 2006 [20] which models real-life driving. According to Yoon et al. [142], in Drive, one needs to reach a goal point by traversing roads. Before applying the move action, one needs to check the light and wait while the light is RED. The chance of having RED is different depending on the road type. There is a slim chance of dying on a wait action. Also, the move action can cause death, depending on the length of the road.
- **Pitchcatch:** This is also problem from the IPPC 2006 [20]. According to Yoon et al. [142], in Pitchcatch, one needs to catch balls of the goal types. Each type can be caught when a pre-defined set of bits is set. Some bits can be set through “setbit” action with a big chance of dying. When a ball is thrown, one can catch or pass it. When caught, all the bits are reset.
- **Elevators-Coins:** This another problem from the IPPC 2006 [20]. According to Yoon et al. [142], in Elevators, one needs to collect coins which are spread among different levels of a building which an elevator can move between. At each level, the agent needs to move to the correct positions to get the coins. The move action is probabilistic and can result in an unwanted position with a 50% chance.
- **Elevators-Management:** This is a problem from the IPPC 2014 [58] where the agent manages several elevators with the goal of shipping passengers that randomly arrive at different floors either to the top or bottom floor as indicated by the passenger. At each step, the agent controls each elevator by either opening its door and indicating the travel direction, by closing the door, and by moving along the indicated direction.
- **Schedule:** This is a problem from the IPPC 2006 [20]. According to Yoon et al. [142], Schedule models a packet scheduling problem where the goal is to process packets with a given goal type. Some types are delivered very rarely and packets are delivered to the queue more frequently than one can process them, and when the queue is overflowed, there is a chance of immediate episode termination.
- **River:** The River problem was proposed by Iain Little and Sylvie Thiébaux [81]. River features only two non-terminal states. The goal of River is to get to another side of a river. From the start state, one could choose to directly swim to the other side. The outcomes of this action are either direct episode termination by drowning or getting to the other side. An alternative is to traverse rocks at a different spot along the river. This action has three outcomes: success, drowning, or landing on an intermediate island from which one can swim to the other side with a high chance of success.
- **Bus Fare:** This is another problem proposed by Iain Little and Sylvie Thiébaux [81] where the goal is to earn enough money for a 3 dollar bus ride. The agent starts with 1 dollar and the episode terminates if the agent reaches 0 dollars at any point. There are always two available actions. The agent can pay one dollar to enter a lottery with a 1% chance of being rewarded the necessary 3 dollars. Alternatively, the agent can enter a different lottery to randomly replace his current cash amount with 1 or 2 dollars.
- **Saving:** Saving is a finite-horizon problem with the aim of maximizing an agent’s money proposed by Hostetler et al. [68]. At each step, the agent has three available actions: *Invest*, *Borrow*, and *Save*. *Borrow* yields an immediate reward of 2 but results in a reward of -3 in n time steps. Before the negative reward is received, the agent cannot borrow anymore. *Save* just yields an immediate reward of 1 without any other effects. *Invest* has no immediate reward but allows the agent to take the *Sell* action within the next m time steps which immediately yields a random but high reward. The *Invest* action cannot be taken again until *Sell* is called or m steps have passed.
- **EV Charging:** This toy problem was defined by Saisubramania et al. [102] and models the problem of optimally charging an electric vehicle. The goal of this problem is to reach a minimum charge level at minimal cost. At each step the agent’s actions are either to charge or discharge at three different speed levels or to do nothing. The cost of each action depends on the current (known) electricity price and the speed of charge. Discharging yields a negative cost. The electricity price changes stochastically and is dependent on the (known) demand level for electricity.
- **8-Puzzle:** 8-Puzzle is played on a 3x3 grid. Each grid cell except one is occupied by an integer. At each step, the agent slides one number adjacent to the empty cell vertically or horizontally into the empty cell. The goal is to reach a predefined goal arrangement of the numbers. Tash and Russell added stochasticity for their experiments by allowing each action to randomly fail [128].

2) Perfect information, continuous action, single-agent problems:

- 1) **Trap:** Trap was first introduced as a toy problem by Cou toux et al. [38] as a minimal example where progressive widening fails. Trap is a two-step game where the (continuous) action space is $A = [0, 1] \subseteq \mathbb{R}$ and starting with $x_0 = 0$, the state is transitioned by $x_{t+1} = x_t + a_t + \mu$, where $\mu \sim \mathcal{U}[0, 1]$ is noise sampled from a uniform distribution. The reward is only dependent on the state and is a piecewise constant function defined as $r(x) = b$ if $x < l$, $r(x) = 0$ if $l \leq x \leq l + w$, and $r(x) = h$ if $x > l + w$ where $b, h, l, w \in \mathbb{R}^+$ are parameters with $h > b$. The region $[l, l + w]$ is the

trap that has to be overcome to get into the high-reward region.

- 2) **Continuous Car:** This problem was introduced as a deterministic test environment by Bianchi et al [16]. The agent controls a car through its acceleration and steering angle, the action space A is $A = [-5, 5] \times [-30, 30] \subseteq \mathbb{R}^2$. The agent's state is a position-velocity pair, whereas the velocity is capped at 20, hence $S = \mathbb{R}^2 \times [0, 20] \subseteq \mathbb{R}^2 \times \mathbb{R}$. Per time step, the car transitions by moving the car in the direction given by the steering angle with the magnitude of the current velocity to which the chosen acceleration is added. The episode ends, when the car either hits a predefined boundary-line or makes it past the goal line which is a vertical line in the plane. The boundaries at Bianchi's instance are defined such that they resemble a curve with a tight bottleneck, the car has to navigate through.
- 3) **Hydrothermal Scheduling:** In this environment which was used by Adrien Coutoux [36], the agent has the task to decide how much each plant in a given set of hydroelectric plants and a thermal power plant contribute to the energy grid such that a non-stationary demand is met. The thermal plant alone can always meet the demand, however, it has a high electricity cost, whereas the hydroelectric energy is cheap. How much power a hydroelectric plant can produce depends on its water level which itself is dependent on a random inflow as well as water coming from other plants. The decision maker's available actions are the amounts of water discharged at each plant, which implicitly determines the energy needed for the thermal plant.

3) *Imperfect information, single-agent problems:*

- **Canadian Traveller Problem:** This problem was originally proposed by Papadimitriou and Yannakakis [94]. The goal is to find the shortest path in a graph between two nodes with the catch that it is not known beforehand which edges are traversable. An edge's traversability is only revealed when the agent reaches one of its adjacent vertices.
- **TargetMonitor:** TargetMonitor was proposed by He et al. [64] and models the problem of a helicopter monitoring objects of interest with the task of detecting whether they are in a region of interest. The objects (representing cars) stochastically move in the plane and are described by their x and y coordinates as well as their orientation θ . The agent itself is described by x, y and z coordinates (z is the altitude) and the agent may move freely in 3d-space. The agent has perfect information about its own coordinates, however, receives only a noisy version of the targets' positions, depending on the agent's altitude as well as the difference between target's and agent's coordinates in the plane. There are predefined, square-shaped regions of interest in the 2d-plane. The goal of TargetMonitor is to decide when each of the monitored objects enters such a region.
- **RockSample:** This problem was proposed by Smith and Simmons [113] which models rover exploration. The rover is placed on an $n \times n$ grid that contains k stones which are either *Good* or *Bad* with equal probability. The rover does not know the status of the stones. The rover may deterministically move to any of the four neighboring cells, examine a stone if the currently occupied tile contains a stone, or sense the *Goodness* of any of the k stones. The sensing yields a noisy version of the stone's *Goodness*. The noise increases exponentially with the sensing distance. If the agent examines a *Good* stone it receives a positive reward and an equally negative reward if the stone was *Bad*. An episode ends with a fixed reward if the rover enters the exit area.
- **PO-Gridworld:** This problem was used by Anthony Stentz for his D* algorithm [121] to model the path-finding problem of a mobile robot with limited sensory information. The name PO-Gridworld which stands for partially observable Gridworld is given by us as Stentz did not name his model. As with Gridworld, PO-Gridworld's states are possible positions on a 2-dimensional grid. Each grid cell is either empty or contains an obstacle. The goal is to reach a certain cell from the current position with minimal cost. In PO-Gridworld there is no stochasticity and the agent may move into any of the eight adjacent cells even when there is an obstacle. However, moving to or from obstacle-occupied cells incurs a significantly higher cost than moving between obstacle-free cells. In contrast to classical Gridworld, in PO-Gridworld the agent is initially only given the position of some obstacles and only observes new obstacles when in an adjacent cell or when within a certain distance.
- **Mobile Robot Navigation:** This is another problem modeling the path-finding problem of a mobile robot. Though they did not explicitly define and name it, this problem was used by Sven Behnke [14]. In Mobile Robot Navigation (MRN), the goal is to control and thus move a robot modeled as a point in $\mathbb{R}^2$ to a goal position with minimal cost. At each step, the robot can move to any position within the vicinity of the current position. The cost of the final path depends on which obstacles (obstacles are disks) and for how long they were crossed. At each step, the agent senses a noisy version of each obstacle's location and size.
- **Hospital Scheduling:** Friha et al. [49] describe a dynamic CSP that we call Hospital Scheduling. The goal of Hospital Scheduling is to assign start times, rooms, and nursing teams to a given set of operations. To account for real-world dynamics, at each time step the duration of an operation may increase, or nursing teams can temporarily become unavailable.

4) *Arcade games:*

- **Pac-Man:** Pac-Man is an arcade game where the player controls a character navigating a maze, consuming pellets while avoiding four enemy ghosts. The objective is to clear the maze by eating all the pellets, with power pellets temporarily

allowing the player to chase and defeat the ghosts.

- **Tetris:** Tetris is a puzzle game that involves manipulating falling tetrominoes to form complete horizontal lines. The game ends when the blocks reach the top of the screen, and players aim to clear as many lines as possible to prevent this.
- **Beamrider:** In Beamrider, the player controls a ship moving along a grid, aiming to destroy enemy objects while avoiding hazards. The game advances through different sectors as the player clears each wave of enemies.
- **Q*bert:** In Q*bert, players control a character that moves across a pyramid of cubes, changing their color by jumping on them. The goal is to change all cubes to a target color while avoiding enemy characters and obstacles.
- **Seaquest:** In Seaquest, the player pilots a submarine, rescuing divers and avoiding underwater enemies such as sharks and hostile submarines. The player must manage limited oxygen by resurfacing periodically while completing objectives.
- **Space Invaders:** In Space Invaders, the player controls a cannon at the bottom of the screen, tasked with eliminating rows of descending alien invaders. The objective is to destroy all invaders before they reach the player's position. The game difficulty increases in the number of opponents and the speed of the game.

5) Board games:

- **Othello:** Othello is a 2-player game played on an 8x8 board where players alternately place a stone of their own color on the board. The goal is to have more stones of one's own color at the end of the game than the opponent.
- **Amazons:** Amazons, proposed by Walter Zamkaskas [145], is played on a 10x10 board where players alternately have to move an Amazon (move pattern like a Queen in chess) and select a tile where the Amazon throws a spear. Thrown spears remain at their position and they restrict the possible moves of other Amazons. The first player to have no legal moves loses.
- **Chess:** Chess is played on an 8x8 board where players alternately move one of its initial 16 pieces from six different types with the goal of capturing the opponent's king.
- **Kriegspiel:** Henry Temple invented Kriegspiel [67] which is a partially observable chess variant where one knows about the position of one's own pieces. The only information a player receives is about captures, checks, and whether there are possible legal pawn captures. Since a player does not see the opponent's pieces, it is not known which moves are legal. If a player tries to perform an illegal move, the player is informed about the illegality of this move and may try again.
- **Go:** Go is typically played on a grid, consisting of 19x19 lines, where two players take turns placing black and white stones on the line intersections. The goal is to capture territory on the board by surrounding the opponent's stones and controlling more space. For research purposes, some authors use simplified versions of Go which are played on smaller boards.

6) Strategy games:

- **Stratega:** Stratega is a general grid-based, unit-based, turn-based strategy game framework [43], [79] with a combinatorial action space. Two commonly used Stratega instances are *Kill the King* where one needs to coordinate warriors, archers, healers, and a king with the goal of eliminating the opponent's king, and *Capture the Flag* where one coordinates warriors only with the goal of stealing a flag from the opponent's base and carrying it to one's own base.
- **Tribes:** Tribes [98] is an open-source reimplementation of the commercial game *The Battle of Polytopia* [1] which is a turn-based, grid-based, stochastic, partially observable strategy game. On a high level, a player controls a tribe by simultaneously managing combat units (e.g. attacking other units or capturing villages), cities (e.g. constructing or destroying buildings), and deciding on global tribe actions (e.g. researching technology) with the goal of either capturing all enemy capitals in the *Capitals* mode or achieving the highest score in the *Score* mode which is gained through various activities.
- **Hero Academy:** Before its shutdown, Hero Academy [47] was a commercial, grid-based, unit-based, turn-based strategy game with a combinatorial action space and the goal of destroying the opponent's crystal or eliminating all its units. Though no longer commercially available the game's rules are simple enough to easily be independently reimplemented.
- **Prismata:** Prismata is a commercial strategy game by Lunarch Studios [123] that features card game and RTS elements. In essence, the player manages an army of units with the goal of destroying all the opponent's units. Unlike Stratega or Hero Academy, Prismata is not grid-based and units do not attack individual units, instead, the decisions are just which abilities to use, what to purchase, which units will be blocking an attack, or which units will be attacked. Attacks themselves are carried out by the game engine and out of the player's control. An action consists of four phases: defense, using abilities, buying units, and breach.

7) Real-time strategy games (RTS):

- **Starcraft:** StarCraft (and its different versions) is a commercial RTS game released by Blizzard Entertainment where players have to build, micro-, and macro-manage an army of units with the goal of destroying all opponent buildings in an imperfect information setting. For research purposes, one usually uses an observable version where the so-called *fog of war* is disabled.

- **SparCraft/JarCraft:** Some research focused on specifically developing agents for combat scenarios in Starcraft. This led to the development of SparCraft, a C++ combat scenario simulator for Starcraft [30]. Justesen reimplemented SparCraft for Java and called it JarCraft [75].
- **RTS combat:** Before they developed SparCraft, Churchill et al. [33] used another RTS combat simulator which they did not name. Even earlier, Sailer et al. [101] also implemented a simple RTS combat simulator. We refer to these as RTS combat.
- **μ -RTS:** Santiago Ontañón [93] developed a minimalistic RTS game specifically for AI research called μ -RTS that encompasses more aspects of RTS games than SparCraft that only simulates combat.
- **RTS Capture the Flag:** Long before SparCraft or μ -RTS were created, Chung et al. [29] implemented their own simple RTS game called Capture The Flag, where the goal is to capture a flag from an enemy’s base and carry it to one’s own base. To achieve this one has to control units in an imperfect information scenario, however, these are the only available actions as there is no resource management of any sort.
- **Wargus:** Wargus [129] is an open-source clone of Warcraft II. In Wargus, one controls either the human or orc race which are functionally identical by micromanaging units, gathering and managing resources to produce new units, build buildings, or unlock new upgrades.

8) Card games:

- **Texas Hold’em Poker:** This Poker variant features four betting rounds. Each player’s hand consists of two private cards and five community cards that are sequentially revealed. In AI research usually the heads-up version is of interest where only two players play. Furthermore, one differentiates between limit and no-limit Poker where in the latter case any-sized bets are allowed while only fixed-sized bets are allowed in the former.
- **Rhode Island Hold’em:** This Poker variant is a slightly simplified version of heads-up Texas Hold’em Poker that was specifically designed for AI research by Shi and Littmann [110]. Rhode Island Hold’em features only three betting rounds, a single private card, and two public cards.
- **Leduc Hold’em Poker:** Leduc Hold’em Poker is an even further simplified heads-up limit Texas Hold’em Poker version than Rhode Island Hold’em also designed for AI research by Southey et al. [116]. This variant has only two betting rounds, a single private card, a single public card, and a greatly reduced deck.
- **Bridge:** Bridge [3] is played with two teams and two players per team. The game consists of three phases. First, each player is dealt 13 private cards out of a 52-card deck. Then, in the bidding phase, the players negotiate who will be the declaring side and its contract which is a number of tricks that have to be won in the subsequent card play phase, and whether there will be a trump suit. Then, in the card play phase, 13 tricks are played, one per card. On each trick, each player plays a card and the trick is won by the highest card of the trick’s suit unless a trump card is played in which case the highest trump card wins. After the play card phase, points are awarded to the declaring side if the contract was met, otherwise to the defending team. A team wins after it has scored a certain number of points.
- **Blackjack:** Blackjack [35] is played with one to eight standard 52-card decks. Initially, each player is dealt two, publicly visible cards and the environment (i.e. the dealer/casino) also receives one public card and one face-down card. The casino or any player may immediately win at this point if they have drawn a hand valued at exactly 21 (a blackjack). If this is not the case, each player has the option to draw additional cards with the goal of achieving a hand valued as close as possible to 21. If after drawing an additional card the hand value exceeds 21, the player immediately loses. After each player has drawn their hands, the dealer reveals the face-down card and then draws additional cards until its hand value reaches a total of 17 or higher. If the dealer exceeds 21 then all players win. Otherwise all players with a hand valued greater than the dealer’s hand win. Throughout the game phases, there are additional decision points such as placing or modifying bets (usually real money in casinos), splitting a hand, or surrendering. Even though Blackjack is a multi-agent game the goal is to beat the environment and not the other players. There exist several Blackjack variants such as Spanish Blackjack that build on this general framework. For computer science research, one may increase the goal of 21 for a longer planning horizon [70] or draw cards from an infinite deck to render any card counting strategies obsolete [69].
- **Hearthstone:** Hearthstone is a digital collectible card game by Activision Blizzard [2] where two players take turns playing cards from their custom-built decks to summon creatures, cast spells, or use weapons. The goal is to reduce the opponent’s health to zero, using a combination of card abilities, strategic planning, and resource management.
- **Doppelkopf:** Doppelkopf is a traditional German trick-taking card game for 4 players, divided into two hidden teams determined by the cards dealt at the start of each round. Team members must deduce their allies during play based on the cards played. Players aim to win tricks containing high-value cards, and the team with the highest cumulative score across rounds wins the game.

B. Paper overview

We compiled all non-learning-based abstraction-using methods that appeared in this survey in Table I.

TABLE I: Table of all non-learning-based abstraction-using methods that appeared in this survey sorted by their **year of publication**. The columns refer to the entire method and not just some subparts. For example, if a method combines a lossless abstraction on top of a lossy abstraction, then the entire method would be marked as not being lossless. Oftentimes, a method is empirically validated on several domains. Sometimes there are additional problem-specific abstractions to make an abstraction feasible on a specific domain. We only consider abstraction techniques as part of the final method if they have been used for all presented experimental domains in the respective paper. By the **state abstraction** column we also implicitly refer to abstractions of belief states (i.e. observation abstraction, as we defined POSGs such that the belief state can be derived from observations). The **domain-specific** column refers to any methods that can only be applied to specific domains (e.g. because they exploit symmetries on board games) or require expert knowledge to be transferred to other domains (e.g. script-based methods). The **solver** column refers to the algorithm that employs the abstraction to derive a decision. For problem-specific modifications of a classical algorithm, we still refer to this as the classical algorithm’s name. For example, extending α - β search to simultaneous move games, we still refer to this as α - β search. The **online abstraction** column refers to methods that intertwine the abstraction construction and its usage such as AS-UCT.

Author	Year of publication	Method name	State abstraction	Action abstraction	Lossless abstraction	Online abstraction	Solver	Domain-specific	Problem domain(s) used in experiments
Xu et al. [140]	2024	SCSA	Yes	Yes	No	Yes	UCT	No	Stratega
Xu et al. [139]	2023	EMCTS	Yes	Yes	No	Yes	UCT	No	Stratega
Bianchi et al. [16]	2022	MCTS-APW2	No	Yes	No	Yes	UCT	No	Continuous Car
Dockhorn et al. [45]	2021	S-MCTS	Yes	No	No	No	UCT	Yes	Stratega
Dockhorn et al. [45]	2021	H-MCTS	Yes	Yes	No	No	UCT	Yes	Stratega
Sokota et al. [115]	2021	Abstraction re-fining	Yes	No	No	Yes	UCT	Yes	Go, Trap
Dockhorn et al. [44]	2021	PRHEA	No	Yes	Yes	No	Rolling Horizon Evolutionary Algorithm (RHEA) [80]	Yes	Stratega
Dockhorn et al. [44]	2021	MO-PRHEA	No	Yes	Yes	No	RHEA	Yes	Stratega
Dockhorn et al. [44]	2021	S-PRHEA	No	Yes	Yes	No	RHEA	Yes	Stratega
Yang and Ontañón [141]	2019	Guided Naïve Sampling	No	Yes	No	No	NaïveMCTS	Yes	μ -RTS
Ontañón and Buro [92]	2019	AHTN-P	No	Yes	No	No	Adversarial Hierarchical Task Network (AHTN) [92]	Yes	μ -RTS
Ontañón and Buro [92]	2019	AHTN-FST	No	Yes	No	No	AHTN	Yes	μ -RTS
Gabor et al. [50]	2019	Subgoal-MCTS	No	Yes	No	Yes	UCT	No	Gridworld, Tetris
Barriga et al. [11], [12]	2015/2018	Puppet search	No	Yes	No	No	UCT/ α - β search	Yes	μ -RTS, Starcraft

Moraes and Lelis [88], [90]	2018	Greedy search	α - β	No	Yes	No	No	Hill-climb search/ α - β	Yes	SparCraft
Moraes and Lelis [88], [90]	2018	Stratified search	α - β	No	Yes	No	No	Hill-climb search/ α - β	Yes	SparCraft
Moraes and Lelis [88], [90]	2018	Greedy Asymmetric Search		No	Yes	No	No	Hill-climb search	Yes	SparCraft
Moraes et al. [87]	2018	Nested Greedy Search		No	Yes	No	No	Hill-climb search	Yes	μ -RTS
Moraes et al. [89], [90]	2018	A1N		No	Yes	No	No	NaïveMCTS	Yes	μ -RTS
Moraes et al. [89], [90]	2018	A2N		No	Yes	No	No	NaïveMCTS	Yes	μ -RTS
Moraes et al. [89], [90]	2018	A3N		No	Yes	No	No	NaïveMCTS	Yes	μ -RTS
Levi Lelis [78]	2017	Stratified Strategy Selection		No	Yes	No	No	Hill-climb search	Yes	SparCraft
Hostetler et al. [68], [69]	2015/ 2017	PARSS		Yes	No	No	Yes	Forward Search Sparse Sampling (FSSS) [133]	No	Saving, Sailing Wind, Racetrack, Spanish Blackjack, Academic Advising, Crossing Traffic, Elevators-Management, Tamarisk, Tetris
Hostetler et al. [69]	2017	rand-FSSS		Yes	No	No	Yes	FSSS	No	Saving, Sailing Wind, Racetrack, Spanish Blackjack, Academic Advising, Crossing Traffic, Elevators-Management, Tamarisk, Tetris
Hostetler et al. [69], [70]	2014/ 2017	χ UCT/AbstractTS	—	Yes	No	No	No	UCT	No	Blackjack
Levi Lelis [78]	2017	SSS^+		No	Yes	No	No	Hill-climb search	Yes	SparCraft
Bai et al. [7]	2016	POMCP(M, ϕ , $\mathcal{O}$)		Yes	Yes	No	No	UCT	Yes	Gridworld
Wang et al. [134]	2016	Portfolio Online Evolution		No	Yes	No	No	Evolutionary Algorithm	Yes	JarCraft
Anand et al. [5]	2016	OGA-UCT		Yes	No	No	Yes	UCT	No	Race Track, SysAdmin, Navigation, Game of Life, Academic Advising

Anand et al. [5]	2016	Pruned UCT	OGA-	Yes	Yes	No	Yes	UCT	No	Race, SysAdmin, Navigation, Game of Life, Academic Advising	Track,
Anand et al. [4]	2015	ASAM-UCT		Yes	No	No	Yes	UCT	No	Sailing Wind, Game of Life, Navigation, Canadian Traveler Problem, Sysadmin	
Anand et al. [4]	2015	ASAP-UCT		Yes	No	No	Yes	UCT	No	Sailing Wind, Game of Life, Navigation, Canadian Traveler Problem, SysAdmin	
Churchill and Buro [32]	2015	Hierarchical Portfolio Search		No	Yes	No	No	UCT/ α - β search	Yes	Prismata	
Balla and Fern [8]	2014	None given		Yes	Yes	No	No	UCT	Yes	Wargus	
Justesen et al. [75]	2014	Script-based UCT		No	Yes	No	No	UCT	Yes	JarCraft	
Justesen et al. [75]	2014	Cluster-based UCT		No	Yes	No	No	UCT	Yes	JarCraft	
Stanescu et al. [117]	2014	Hierarchical Adversarial Search		Yes	Yes	No	No	α - β search	Yes	SparCraft	
Jiang et al. [71]	2014	AS-UCT		Yes	No	No	Yes	UCT	No	Sailing Wind, Rock-Sample, Othello	
Churchill and Buro [31]	2013	Portfolio Greedy Search		No	Yes	No	No	Hill-climb search	Yes	SparCraft	
Adrien Couetoux [36], [37]	2012	BlindValue		No	Yes	No	Yes	UCT	No	Hydrothermal Scheduling	
Churchill et al. [33]	2012	None given		No	Yes	No	No	α - β search	Yes	RTS combat	
Adrien Couetoux [36], [38]	2011	Double Progressive Widening		Yes	Yes	No	Yes	UCT	No	Trap	
Uriarte et al. [131]	2009	None given		Yes	Yes	No	No	UCT/ α - β search	Yes	Starcraft	
Yoon et al. [143]	2008	FF-Hindsight		No	Yes	No	No	FF [66]	No	Climber, River, Bus-fare, Tireworld	
Lorentz [84]	2008	InvaderMC		No	Yes	Yes	No	UCT	Yes	Amazons	
Childs et al. [28]	2008	None given		No	Yes	Yes	No	UCT	Yes	Go	
Wang et al. [135]	2008	UCB-AIR or MCTS-APW		No	Yes	No	Yes	UCT	No	<i>No experiments</i>	

Saito et al. [104]	2007	Alternating-Layer UCT	No	Yes	Yes	No	UCT	Yes	Go
Sailer et al. [101]	2007	None Given	No	Yes	No	No	Linear Programming	Yes	RTS Combat
Ciancarini Favini [34]	2007	Darkboard	Yes	Yes	No	No	MaxiMax [34]	Yes	Kriegspiel
Yoon et al. [142]	2007	FF-Replan	No	Yes	No	No	FF	No	Blocksworld, Boxworld, Fileworld, Tireworld, Towers of Hanoise, Zeno travel, Drive, Pitchcatch, Elevators-Coins, Schedule
Bouzy [22]	2006	Set Pruning	No	Yes	No	Yes	Progressive Pruning (PP) [23]	No	Go
Chung et al. [29]	2005	MCPlan	Yes	Yes	No	No	Monte Carlo Search	Yes	RTS Capture the Flag
Sven Behnke [14]	2003	None given	Yes	Yes	No	No	A* [62]	Yes	Mobile Robot Navigation
Friha et al. [49]	1997	DISA	Yes	Yes	Yes	No	An unnamed constraint satisfaction problem solver	Yes	Hospital Scheduling
Matthew Ginsberg [56]	1996	Partition Search	Yes	No		Yes	α - β -search	Yes	Bridge
Anthony Stentz [120], [121]	1993	D*	Yes	No	No	No	A*	Yes	PO-Gridworld